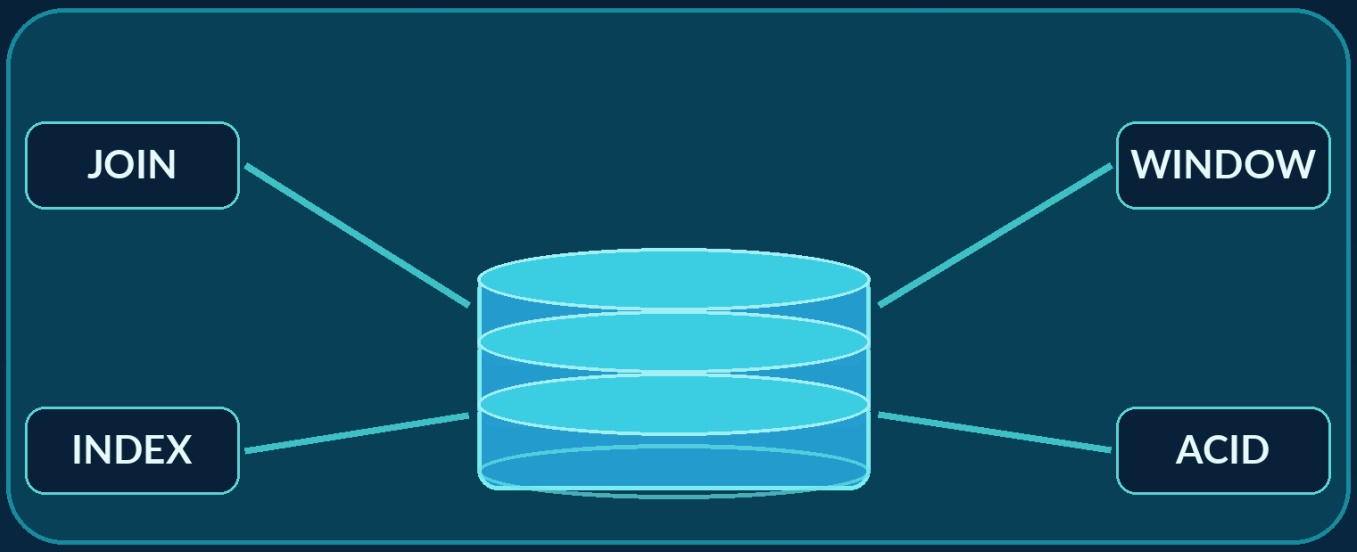
PREMIUM HANDWRITTEN NOTES EDITION

SQL

ZERO TO HERO

INTERVIEW GUIDE

Clear answers. Proven queries.
Visual explanations. Real results.



- 100 ORIGINAL INTERVIEW QUESTIONS
- 100 ANSWERED FOLLOW-UPS
- 15 SQL LABS + VERIFIED RESULTS
- POSTGRESQL 17 | MYSQL 8.4
- 20 CHAPTERS | 30-DAY PLAN

SQL FUNDAMENTALS / QUERY TUNING / SYSTEM DESIGN

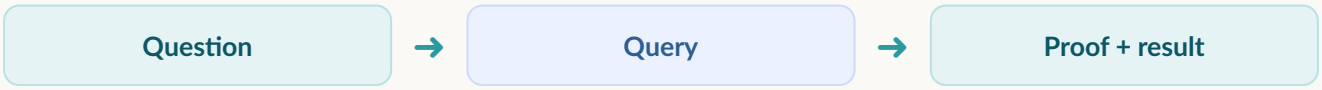
START HERE

Your SQL interview notebook

Understand the question first, then predict the result. Learn why the query works and what breaks when data is NULL, duplicated or concurrent.

THE METHOD

Say what one row represents. Read the short answer. Trace the SQL. Predict the rows. Explain the edge case. Finish with the follow-up.



WHAT YOU GET

100 original question wordings • 100 individually answered follow-ups • 20 chapters • 15 verified worked SQL exercises • shared training database • PostgreSQL 17 / MySQL 8.4 notes • 30-day practice route.

IMPORTANT ENGINE BOUNDARY

Portable SELECT results can be tested with SQLite. Locking, isolation, DDL behavior and execution plans must be tested on the named PostgreSQL or MySQL engine.

COMPLETE COVERAGE

20 chapters • 100 questions

Question wording matches the supplied guide; you can cross-reference by question number.

01 / Q001–Q005	SQL foundations and query reasoning
02 / Q006–Q010	Types, NULL and three-valued logic
03 / Q011–Q015	Schema, constraints and DDL
04 / Q016–Q020	Filtering, dates and deterministic ordering
05 / Q021–Q025	Joins and result cardinality
06 / Q026–Q030	Aggregation and grouping
07 / Q031–Q035	Subqueries, EXISTS and null-safe anti-joins
08 / Q036–Q040	CTEs and recursive queries
09 / Q041–Q045	Window functions
10 / Q046–Q050	Common interview query challenges

COMPLETE COVERAGE

Chapters 11–20

From safe writes and indexing to transaction isolation and senior SQL design.

11 / Q051–Q055	Data changes, upsert and deduplication
12 / Q056–Q060	Normalization and schema design
13 / Q061–Q065	Indexes and access-path design
14 / Q066–Q070	Execution plans and query tuning
15 / Q071–Q075	Transactions and ACID
16 / Q076–Q080	Isolation and MVCC
17 / Q081–Q085	Locks, deadlocks and concurrency control
18 / Q086–Q090	Views, routines, partitioning and maintenance
19 / Q091–Q095	Application SQL, security and pagination
20 / Q096–Q100	Senior scenarios and query-review discipline

SQL DIALECTS

PostgreSQL vs MySQL

Portable concepts are shared; important database guarantees are not always the same.

Topic	PostgreSQL 17	MySQL 8.4 / InnoDB
Default isolation	Read Committed	Repeatable Read
Full outer join	Native FULL OUTER JOIN	No native FULL OUTER JOIN
Upsert	ON CONFLICT	ON DUPLICATE KEY UPDATE
TRUNCATE rollback	Can roll back within transaction	Implicit commit
Storage	Heap + separate indexes	Clustered primary key rows
Query plan	EXPLAIN (ANALYZE, BUFFERS)	EXPLAIN ANALYZE
Date interval	INTERVAL '7 days'	INTERVAL 7 DAY

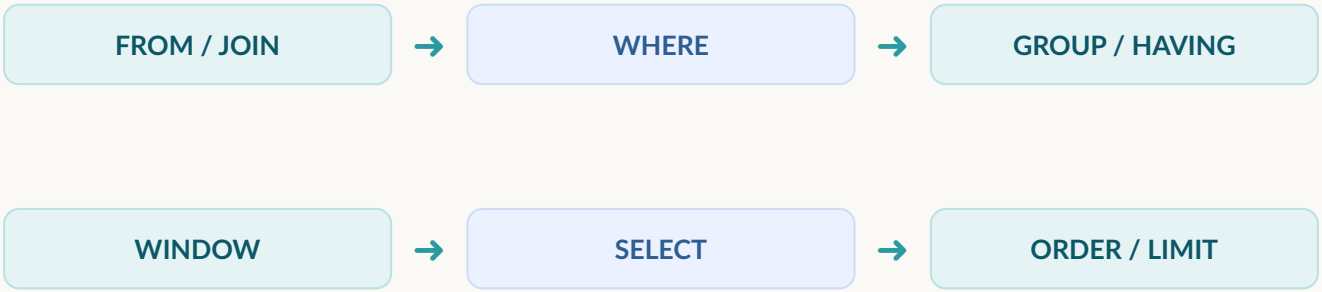
INTERVIEW REMINDER

Name the database and isolation level before making locking or persistence promises.

MENTAL MODEL

Prove every query result

The order below is a logical reasoning aid, not the physical plan chosen by the optimizer.



ALWAYS DEFINE THE GRAIN

Is one output row an employee, order, customer, joined detail row, or aggregate group? This decides whether joins duplicate facts.

COUNTEREXAMPLE: JOIN MULTIPLICATION

Order 101 has two item rows and header amount 100. A detail join produces two copies of the header. Summing the joined header amount gives 200 instead of 100.

CHAPTER 01 • SQL FOUNDATIONS AND QUERY REASONING

QUESTION Q001

What is SQL and how is it different from procedural code?

SHORT INTERVIEW ANSWER

SQL describes the required data result; the database optimizer chooses an execution plan.

TRACE THE EXAMPLE

```
SELECT employee_id, name FROM employees WHERE salary >= 80000;
```

Input rows → Filter / project → Result grain

WHY THIS WORKS

The query specifies columns and eligible rows rather than a loop through storage pages. A relational result can contain duplicate rows unless the query removes them. Explain what one result row represents before writing joins.

WATCH OUT

Declarative syntax does not guarantee an efficient plan.

FOLLOW-UP QUESTION + ANSWER

What is the grain of this result?

A row represents one employee whose known salary is at least 80,000. employee_id identifies the row.

CHAPTER 01 • SQL FOUNDATIONS AND QUERY REASONING

QUESTION Q002

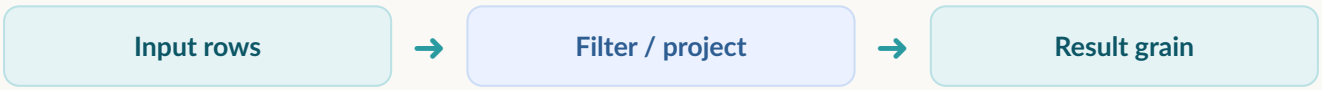
What are DDL, DML, DQL, DCL and TCL?

SHORT INTERVIEW ANSWER

They are common teaching categories for defining structures, changing/querying data, controlling privileges and managing transactions.

TRACE THE EXAMPLE

```
DDL: CREATE, ALTER, DROP
DML: INSERT, UPDATE, DELETE; DQL label: SELECT
DCL: GRANT, REVOKE; TCL: COMMIT, ROLLBACK, SAVEPOINT
```



WHY THIS WORKS

The DQL/DML classification of SELECT varies by teaching convention. Learn command behavior rather than treating the categories as a portable transactional specification. DDL commit and rollback behavior differs by database.

WATCH OUT

Saying all DDL always commits is incorrect across engines.

FOLLOW-UP QUESTION + ANSWER

How does MySQL DDL differ from PostgreSQL DDL?

PostgreSQL can roll back many supported DDL operations inside a transaction; MySQL commonly performs implicit commits for DDL, including TRUNCATE. Check the exact command before migration.

CHAPTER 01 • SQL FOUNDATIONS AND QUERY REASONING

QUESTION Q003

What is a relational key?

SHORT INTERVIEW ANSWER

A key identifies a row according to an invariant; candidate keys are minimal identifiers, and one can be selected as the primary key.

TRACE THE EXAMPLE

```
employees.employee_id is primary key
An enforced unique business identifier can be another candidate key
```

Input rows → Filter / project → Result grain

WHY THIS WORKS

A surrogate ID simplifies references but does not enforce business uniqueness by itself. Use appropriate unique constraints for identifiers that must not repeat. Composite keys combine columns when the identity requires them.

WATCH OUT

A generated ID does not prevent duplicate business records.

FOLLOW-UP QUESTION + ANSWER

Which business uniqueness constraint would you add?

For accounts, enforce UNIQUE on the immutable account number (and NOT NULL). For tenant-scoped IDs, consider UNIQUE(tenant_id, external_id).

CHAPTER 01 • SQL FOUNDATIONS AND QUERY REASONING

QUESTION Q004

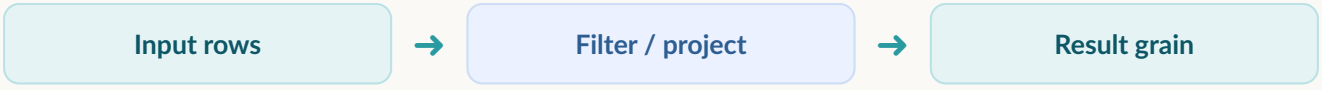
What is the logical processing order of a SELECT?

SHORT INTERVIEW ANSWER

Reason roughly from FROM/JOIN to WHERE, GROUP BY, HAVING, window evaluation, projection/distinct, ORDER BY and limiting, subject to SQL semantics.

TRACE THE EXAMPLE

```
FROM -> WHERE -> GROUP BY -> HAVING
Window calculations -> SELECT / DISTINCT -> ORDER BY -> LIMIT
```



WHY THIS WORKS

This is a reasoning model, not a promise about the physical operator schedule. The optimizer can reorder equivalent operations. Alias visibility and supported clauses vary by dialect, so use a subquery when a derived result must be filtered.

WATCH OUT

Physical execution is not a literal top-to-bottom reading of SQL.

FOLLOW-UP QUESTION + ANSWER

Why cannot WHERE normally use a window result?

WHERE filters rows before window values are calculated in the logical query model. Calculate the window in a CTE or subquery, then filter outside.

CHAPTER 01 • SQL FOUNDATIONS AND QUERY REASONING

QUESTION Q005

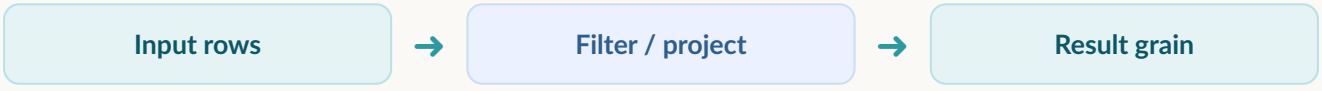
Does SQL return rows in insertion order?

SHORT INTERVIEW ANSWER

No. A required order needs ORDER BY, including a deterministic tie-breaker when pagination or ranking needs it.

TRACE THE EXAMPLE

```
SELECT employee_id, salary FROM employees
ORDER BY salary DESC, employee_id ASC;
```



WHY THIS WORKS

Indexes and current plans can make an unordered result appear stable, but that is not a contract. Null placement differs by database/defaults; define it explicitly when required. Ties need additional ordering for exact row selection.

WATCH OUT

LIMIT without ORDER BY is not a reliable top-N query.

FOLLOW-UP QUESTION + ANSWER

How would you order null salaries last portably?

ORDER BY CASE WHEN salary IS NULL THEN 1 ELSE 0 END, salary DESC, employee_id. The CASE places missing salaries last and ID breaks ties.

CHAPTER 02 • TYPES, NULL AND THREE-VALUED LOGIC

QUESTION Q006

How is NULL different from zero or an empty string?

SHORT INTERVIEW ANSWER

NULL represents an absent/unknown value; zero and an empty string are actual values under the chosen database semantics.

TRACE THE EXAMPLE

```
SELECT employee_id FROM employees WHERE salary IS NULL;
```

Known / unknown → 3-valued logic → Correct row

WHY THIS WORKS

A normal equality comparison to NULL produces unknown rather than true. WHERE keeps rows where its predicate is true. Use IS NULL/IS NOT NULL and decide whether missing data is valid in the domain.

WATCH OUT

salary = NULL does not find null salaries.

FOLLOW-UP QUESTION + ANSWER

What does NOT(salary = 1000) do for NULL?

NULL = 1000 is UNKNOWN, and NOT UNKNOWN is still UNKNOWN. WHERE therefore excludes that row.

CHAPTER 02 • TYPES, NULL AND THREE-VALUED LOGIC

QUESTION Q007

What is SQL three-valued logic?

SHORT INTERVIEW ANSWER

Predicates can evaluate to true, false or unknown, and unknown affects filtering and Boolean combinations.

TRACE THE EXAMPLE

```
TRUE AND UNKNOWN -> UNKNOWN
FALSE AND UNKNOWN -> FALSE
TRUE OR UNKNOWN -> TRUE
```

Known / unknown → 3-valued logic → Correct row

WHY THIS WORKS

Unknown commonly comes from comparisons involving NULL. Do not assume programming-language Boolean rules with a null simply replaced by false. A CHECK constraint can accept unknown, so NOT NULL may be required separately.

WATCH OUT

CHECK(salary > 0) alone does not reject NULL salary.

FOLLOW-UP QUESTION + ANSWER

How do you require a positive known salary?

Use salary DECIMAL(12,2) NOT NULL CHECK (salary > 0). NOT NULL rejects unknown, while CHECK rejects nonpositive values.

CHAPTER 02 • TYPES, NULL AND THREE-VALUED LOGIC

QUESTION Q008

How do COALESCE and NULLIF help?

SHORT INTERVIEW ANSWER

COALESCE chooses the first non-null expression; NULLIF returns NULL when its arguments compare equal.

TRACE THE EXAMPLE

```
SELECT COALESCE(salary, 0) FROM employees;
SELECT 100.0 / NULLIF(denominator, 0) FROM calculation;
```

Known / unknown → 3-valued logic → Correct row

WHY THIS WORKS

Substitution is a business choice: unknown salary is not automatically zero. NULLIF can avoid a division-by-zero failure by producing a null result. Compatible types and engine arithmetic behavior still matter.

WATCH OUT

A null ratio and a zero ratio have different meanings.

FOLLOW-UP QUESTION + ANSWER

What should the report show for an undefined denominator?

Return NULL or an explicit 'not applicable' indicator, not a made-up zero. Document the rule for consumers.

CHAPTER 02 • TYPES, NULL AND THREE-VALUED LOGIC

QUESTION Q009

Why use DECIMAL/NUMERIC rather than floating point for money?

SHORT INTERVIEW ANSWER

Exact decimal types avoid binary floating-point approximation for the supported precision and scale.

TRACE THE EXAMPLE

```
amount DECIMAL(12,2)
-- 12 total digits, 2 fractional digits
```

Known / unknown → 3-valued logic → Correct row

WHY THIS WORKS

Specify rounding, currency and permitted range. Exact storage does not decide business rounding or stop arithmetic overflow automatically. Integer minor units are another deliberate option; not every currency uses two decimal places.

WATCH OUT

FLOAT is not an exact currency ledger type.

FOLLOW-UP QUESTION + ANSWER

How would you model currencies with different scales?

Store currency code and amount using a defined decimal scale or integer minor units according to each currency. Convert and round with an explicit policy.

CHAPTER 02 • TYPES, NULL AND THREE-VALUED LOGIC

QUESTION Q010

How do DATE and timestamp types differ?

SHORT INTERVIEW ANSWER

DATE stores a calendar date; timestamp types store time-of-day, with engine-specific time-zone semantics.

TRACE THE EXAMPLE

```
-- PostgreSQL: timestamp without time zone vs timestamptz
-- MySQL: DATETIME vs TIMESTAMP have different conversion/range rules
```

Known / unknown → 3-valued logic → Correct row

WHY THIS WORKS

Choose whether the value is a local civil time, an instant, or a date. Session time zone affects some displays/conversions. Keep inclusive start/exclusive end bounds for time intervals, and define the business reporting zone.

WATCH OUT

Timestamp type names are not portable semantic guarantees.

FOLLOW-UP QUESTION + ANSWER

How do you report one day across a daylight-saving change?

Define the business time zone; construct local midnight-to-next-midnight bounds in that zone, convert both to instants and query a half-open interval.

CHAPTER 03 • SCHEMA, CONSTRAINTS AND DDL

QUESTION Q011

How do PRIMARY KEY, UNIQUE and NOT NULL differ?

SHORT INTERVIEW ANSWER

Primary key identifies each row and disallows null key parts; UNIQUE enforces uniqueness under engine rules; NOT NULL forbids absence.

TRACE THE EXAMPLE

```
CREATE TABLE tags (
  tag_id INTEGER PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
);
```

Business rule → Constraint check → Valid write

WHY THIS WORKS

A table can have multiple unique constraints but one primary key declaration. NULL handling in unique constraints differs by database/options. Do not infer that UNIQUE alone always makes a column a non-null business identifier.

WATCH OUT

Nullable unique behavior is not identical to a primary key.

FOLLOW-UP QUESTION + ANSWER

How do you require one unique known email?

Use email VARCHAR(...) NOT NULL with a UNIQUE constraint; decide whether normalized/case-insensitive uniqueness is needed and implement it for your engine.

CHAPTER 03 • SCHEMA, CONSTRAINTS AND DDL

QUESTION Q012

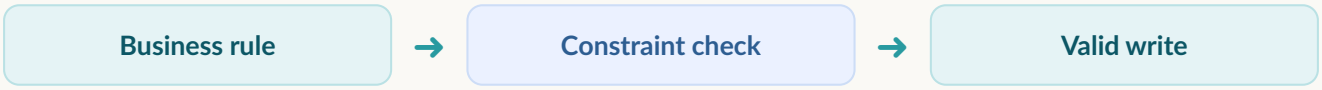
What does a foreign key enforce?

SHORT INTERVIEW ANSWER

It requires referenced identity to exist for applicable referencing values and constrains related modifications through its action policy.

TRACE THE EXAMPLE

```
FOREIGN KEY (department_id) REFERENCES departments(department_id)
```



WHY THIS WORKS

Nullable foreign keys can represent optional relationships under the configured match semantics. Deletion actions such as RESTRICT or CASCADE have business consequences. PostgreSQL does not automatically create a referencing-column index; MySQL InnoDB has index requirements/creation behavior.

WATCH OUT

A foreign key does not automatically prove a query's join is one-to-one.

FOLLOW-UP QUESTION + ANSWER

Which side can have many rows?

The referencing/child table can contain many rows pointing to one parent. For example, many employees can reference one department.

CHAPTER 03 • SCHEMA, CONSTRAINTS AND DDL

QUESTION Q013

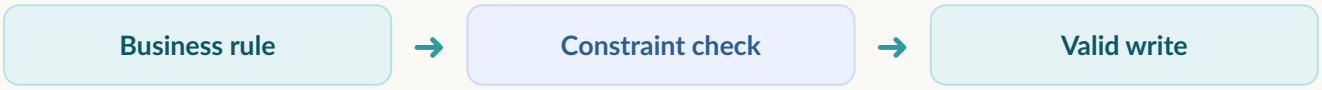
When should you use a CHECK constraint?

SHORT INTERVIEW ANSWER

Use it for supported row-level validity rules, along with required nullability constraints.

TRACE THE EXAMPLE

```
quantity INTEGER NOT NULL CHECK (quantity > 0)
```



WHY THIS WORKS

A check can validate a row's values but is not a general cross-row aggregate invariant. Expressions and enforcement differ by engine/version. A uniqueness constraint or proper transaction protocol is needed for concurrent cross-row rules.

WATCH OUT

A positive check without NOT NULL can permit unknown values.

FOLLOW-UP QUESTION + ANSWER

How would you enforce one active booking per slot?

Enforce a uniqueness rule on the slot identity for active bookings: in PostgreSQL a partial unique index may work; otherwise use a modeled active-slot claim with a unique key and transaction.

CHAPTER 03 • SCHEMA, CONSTRAINTS AND DDL

QUESTION Q014

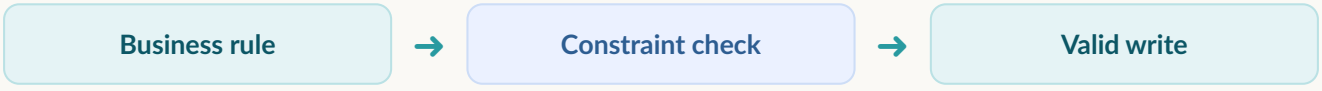
How do DELETE, TRUNCATE and DROP differ?

SHORT INTERVIEW ANSWER

DELETE removes selected rows; TRUNCATE removes table contents through a different engine operation; DROP removes the object definition.

TRACE THE EXAMPLE

```
DELETE FROM demo_rows WHERE id = 1;
-- TRUNCATE demo_rows; DROP TABLE demo_rows; use only disposable fixtures.
```



WHY THIS WORKS

Do not summarize them only as slow/fast. PostgreSQL TRUNCATE can be rolled back in a transaction, while MySQL TRUNCATE causes an implicit commit. Locking, triggers, identity behavior and foreign-key restrictions differ.

WATCH OUT

TRUNCATE rollback behavior is not portable.

FOLLOW-UP QUESTION + ANSWER

Which operation preserves the table schema?

DELETE and TRUNCATE preserve the table definition; DROP TABLE removes it. Other behavior differs by engine.

CHAPTER 03 • SCHEMA, CONSTRAINTS AND DDL

QUESTION Q015

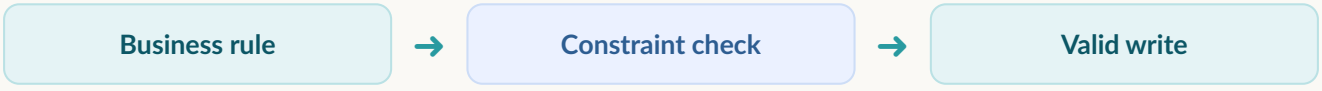
How do you change a large production table safely?

SHORT INTERVIEW ANSWER

Plan compatibility, lock duration, backfill load, constraints and rollback around the specific engine operation.

TRACE THE EXAMPLE

```
Add nullable column -> deploy compatible code
Backfill in bounded batches -> validate -> enforce required constraint
```



WHY THIS WORKS

Online/concurrent options have engine-specific limitations and still consume resources. Avoid assuming every ALTER is instant. Mixed old/new application versions must tolerate the transitional schema. Test on representative volume and inspect locks.

WATCH OUT

A migration that works on ten rows can block a large live table.

FOLLOW-UP QUESTION + ANSWER

What evidence would you collect before rollout?

Collect row counts, disk and index sizes, traffic, lock duration, rollback plan, representative rehearsal timings and old/new app compatibility evidence.

CHAPTER 04 • FILTERING, DATES AND DETERMINISTIC ORDERING

QUESTION Q016

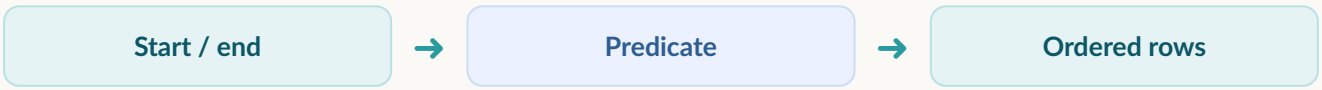
How do WHERE, BETWEEN, IN and LIKE differ?

SHORT INTERVIEW ANSWER

They express general predicates, inclusive bounds, membership and pattern matching respectively.

TRACE THE EXAMPLE

```
WHERE salary BETWEEN 60000 AND 90000
WHERE department_id IN (10,20)
WHERE name LIKE 'M%'
```



WHY THIS WORKS

BETWEEN includes both ends, which can be awkward for timestamp ranges. LIKE case sensitivity follows collation/type rules rather than one universal SQL behavior. Escape literal wildcard characters when needed.

WATCH OUT

BETWEEN day-start and next-day-start includes the next boundary.

FOLLOW-UP QUESTION + ANSWER

How would you filter exactly one month?

Use >= the first instant of the month and < the first instant of the next month. Bind both boundaries with the reporting time zone.

CHAPTER 04 • FILTERING, DATES AND DETERMINISTIC ORDERING

QUESTION Q017

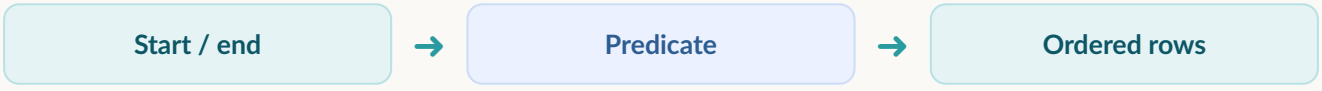
Why use a half-open timestamp range?

SHORT INTERVIEW ANSWER

It includes the start and excludes the next boundary without guessing the last representable instant.

TRACE THE EXAMPLE

```
WHERE ordered_at >= :start
  AND ordered_at < :next_start
```



WHY THIS WORKS

Bind timestamps derived in the intended business zone. This can also be more index-friendly than wrapping the column in a date-extraction function. Account for timestamp precision and session conversion behavior.

WATCH OUT

'23:59:59' can miss fractional-second rows.

FOLLOW-UP QUESTION + ANSWER

How do you build month bounds correctly?

Compute the first day of this month and the first day of the following month in the business zone; use start inclusive and next-start exclusive.

CHAPTER 04 • FILTERING, DATES AND DETERMINISTIC ORDERING

QUESTION Q018

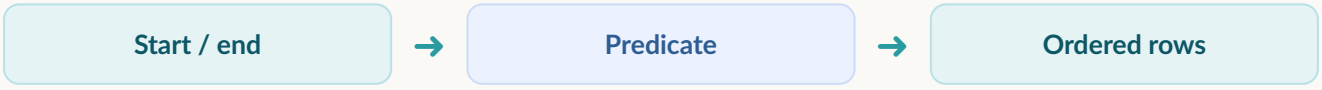
How do CASE expressions help in reporting?

SHORT INTERVIEW ANSWER

CASE maps conditions to values while keeping the rule inside a set-based query.

TRACE THE EXAMPLE

```
SELECT employee_id,
 CASE WHEN salary IS NULL THEN 'UNKNOWN'
      WHEN salary >= 90000 THEN 'HIGH' ELSE 'OTHER' END AS band
FROM employees;
```



WHY THIS WORKS

Branches are considered in order and the resulting types must be compatible. Explicit handling of NULL prevents unknown values from being mislabeled. Use CASE for conditional aggregation as well as display values.

WATCH OUT

A label does not replace the original value's semantics.

FOLLOW-UP QUESTION + ANSWER

What happens when two conditions overlap?

CASE returns the value from the first matching WHEN branch. Order overlapping conditions from most specific to general.

CHAPTER 04 • FILTERING, DATES AND DETERMINISTIC ORDERING

QUESTION Q019

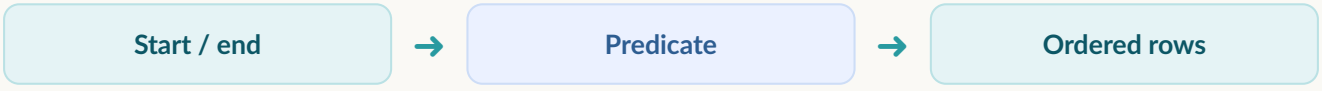
How do you paginate deterministically with LIMIT?

SHORT INTERVIEW ANSWER

Apply an explicit stable ordering with a unique tie-breaker; then choose offset or seek behavior deliberately.

TRACE THE EXAMPLE

```
SELECT order_id, ordered_at FROM orders
ORDER BY ordered_at DESC, order_id DESC LIMIT 20 OFFSET 40;
```



WHY THIS WORKS

LIMIT/OFFSET syntax works in the main PostgreSQL/MySQL examples. Other engines use alternatives. Inserts/deletes between requests can shift offset pages; stable ordering alone does not provide a cross-request snapshot.

WATCH OUT

Deep offsets still require work on preceding rows.

FOLLOW-UP QUESTION + ANSWER

When would seek pagination be better?

Seek/keyset pagination is better for deep sequential browsing with a stable unique sort key; it avoids scanning large OFFSET prefixes and reduces page drift.

CHAPTER 04 • FILTERING, DATES AND DETERMINISTIC ORDERING

QUESTION Q020

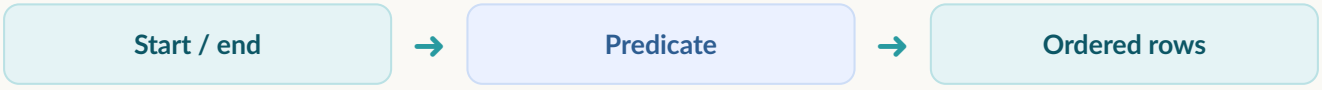
How do you write portable date arithmetic?

SHORT INTERVIEW ANSWER

Do not assume interval syntax is identical; label the engine and bind computed bounds when useful.

TRACE THE EXAMPLE

```
-- PostgreSQL:
SELECT CURRENT_TIMESTAMP - INTERVAL '7 days';
-- MySQL:
SELECT CURRENT_TIMESTAMP - INTERVAL 7 DAY;
```



WHY THIS WORKS

Functions such as date truncation, date differences and formatting differ. Parameterized start/end timestamps often keep application queries simple and consistent. Define whether seven days means elapsed hours or calendar days.

WATCH OUT

Calendar arithmetic can differ from fixed elapsed seconds.

FOLLOW-UP QUESTION + ANSWER

Which time zone controls the report?

Use the business/reporting time zone selected by the product contract, not an accidental database session default.

CHAPTER 05 • JOINS AND RESULT CARDINALITY

QUESTION Q021

How do INNER and LEFT JOIN differ?

SHORT INTERVIEW ANSWER

INNER JOIN returns matching pairs; LEFT JOIN also preserves unmatched left rows with null-extended right columns.

TRACE THE EXAMPLE

```
SELECT d.name, e.name FROM departments d
LEFT JOIN employees e ON e.department_id = d.department_id;
```

Left relation → Match keys → Output rows

WHY THIS WORKS

A department with no employees still appears in the left-join result. A department with many employees appears once for every matching employee. Understand the keys and multiplicity before aggregating.

WATCH OUT

LEFT JOIN does not guarantee one result row per left row.

FOLLOW-UP QUESTION + ANSWER

How many rows does Engineering produce here?

Engineering has three matching employees: Sai, Meena and Arun. LEFT JOIN returns one row per match.

CHAPTER 05 • JOINS AND RESULT CARDINALITY

QUESTION Q022

Why can a WHERE predicate turn a LEFT JOIN into inner-like behavior?

SHORT INTERVIEW ANSWER

A predicate requiring a non-null right-side match discards null-extended rows after the join.

TRACE THE EXAMPLE

```
LEFT JOIN employees e ON e.department_id=d.department_id
WHERE e.salary > 80000
```

Left relation → Match keys → Output rows

WHY THIS WORKS

If the requirement is preserve every department while matching only high earners, place the right-side condition in ON. If the requirement is departments with a high earner, filtering may be intentional. Explain the requirement first.

WATCH OUT

Moving a condition between ON and WHERE can change semantics.

FOLLOW-UP QUESTION + ANSWER

How do you preserve empty departments?

Move the right-side salary condition into JOIN ... ON and preserve each department; count employee_id to get zero for unmatched departments.

CHAPTER 05 • JOINS AND RESULT CARDINALITY

QUESTION Q023

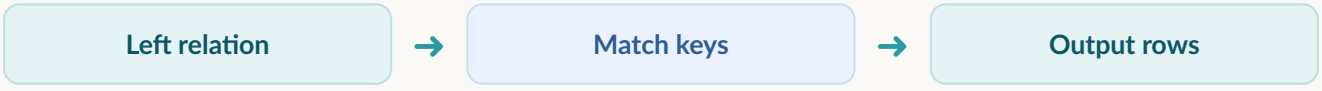
Why do joins multiply rows?

SHORT INTERVIEW ANSWER

Each matching pair produces a row; multiple one-to-many relationships can create a multiplication of combinations.

TRACE THE EXAMPLE

```
Order has 2 items and 3 payments
Joining both detail tables directly can create 6 rows
```



WHY THIS WORKS

Declare the intended grain and aggregate each detail relation to that grain before joining. DISTINCT on final values can hide symptoms but cannot generally repair duplicated totals. Validate constraints rather than assuming uniqueness.

WATCH OUT

SUM(DISTINCT amount) also collapses genuinely equal amounts.

FOLLOW-UP QUESTION + ANSWER

How would you avoid doubled order revenue?

Aggregate order-items to order_id before joining, or sum header amounts from the orders table directly. Never fix duplicated headers with SUM(DISTINCT amount).

CHAPTER 05 • JOINS AND RESULT CARDINALITY

QUESTION Q024

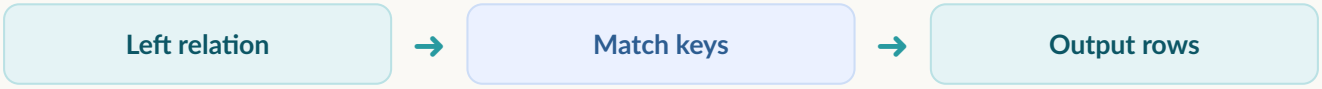
What are self joins and cross joins?

SHORT INTERVIEW ANSWER

A self join relates rows of one table through aliases; a cross join produces every pair of rows from its inputs.

TRACE THE EXAMPLE

```
SELECT e.name, m.name AS manager FROM employees e
LEFT JOIN employees m ON m.employee_id=e.manager_id;
```



WHY THIS WORKS

Use clear aliases for employee/manager roles. A cross join can intentionally generate a calendar/product matrix, but an accidental missing join condition can explode row counts. Multiply input cardinalities to estimate the result.

WATCH OUT

Sharing a table name does not remove the need for a relationship condition.

FOLLOW-UP QUESTION + ANSWER

How would you find employees without a manager?

LEFT JOIN employees m ON m.employee_id=e.manager_id WHERE e.manager_id IS NULL; this includes top-level and unassigned employees.

CHAPTER 05 • JOINS AND RESULT CARDINALITY

QUESTION Q025

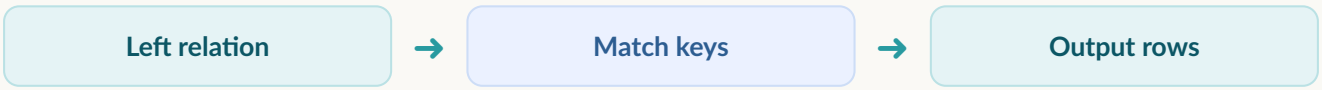
Does MySQL support FULL OUTER JOIN like PostgreSQL?

SHORT INTERVIEW ANSWER

MySQL 8.4 does not provide native FULL OUTER JOIN; PostgreSQL does. Emulation needs careful duplicate and match handling.

TRACE THE EXAMPLE

```
-- PostgreSQL:
SELECT * FROM a FULL OUTER JOIN b ON a.id=b.id;
-- MySQL: left branch UNION ALL right-only branch with NOT EXISTS.
```



WHY THIS WORKS

Keep unmatched rows from both sides without returning matched rows twice. Use explicit columns and a non-null key or null-safe anti-match policy. Many interview questions do not require a full join, so verify the actual goal.

WATCH OUT

UNION can unintentionally remove legitimate duplicate rows in an emulation.

FOLLOW-UP QUESTION + ANSWER

How do you isolate the right-only branch?

Use the left-join rows from B with no match in A, commonly WHERE a.id IS NULL when the left key is non-null; add them with UNION ALL.

CHAPTER 06 • AGGREGATION AND GROUPING

QUESTION Q026

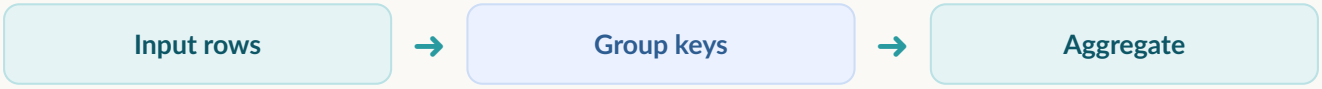
How do GROUP BY and DISTINCT differ?

SHORT INTERVIEW ANSWER

GROUP BY forms groups for aggregate reasoning; DISTINCT removes duplicate projected result rows.

TRACE THE EXAMPLE

```
SELECT DISTINCT department_id FROM employees;
SELECT department_id, COUNT(*) FROM employees GROUP BY department_id;
```



WHY THIS WORKS

They can sometimes produce the same columns but express different intent. Choose grouping keys representing the report grain. PostgreSQL/MySQL have specific functional-dependency rules; avoid relying on arbitrary ungrouped columns.

WATCH OUT

Disabling strict group rules can yield misleading selected values.

FOLLOW-UP QUESTION + ANSWER

Which columns define one report row?

Choose grouping columns that uniquely define the report entity: for example, customer_id for one row per customer.

CHAPTER 06 • AGGREGATION AND GROUPING

QUESTION Q027

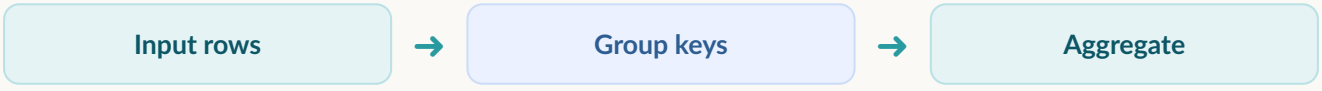
How do COUNT(*), COUNT(column) and COUNT(DISTINCT column) differ?

SHORT INTERVIEW ANSWER

They count rows, non-null column values and distinct non-null values respectively.

TRACE THE EXAMPLE

```
SELECT COUNT(*), COUNT(salary), COUNT(DISTINCT salary) FROM employees;
-- Fixture result: 6, 5, 4
```



WHY THIS WORKS

Null salaries contribute to row count but not salary-value counts. With a left join, COUNT(*) also counts a preserved row for an empty department; count a non-null employee key instead.

WATCH OUT

COUNT(*) after LEFT JOIN can report one employee for an empty group.

FOLLOW-UP QUESTION + ANSWER

How do you count zero employees correctly?

SELECT d.department_id, COUNT(e.employee_id) FROM departments d LEFT JOIN employees e ON e.department_id=d.department_id GROUP BY d.department_id; empty HR returns zero.

CHAPTER 06 • AGGREGATION AND GROUPING

QUESTION Q028

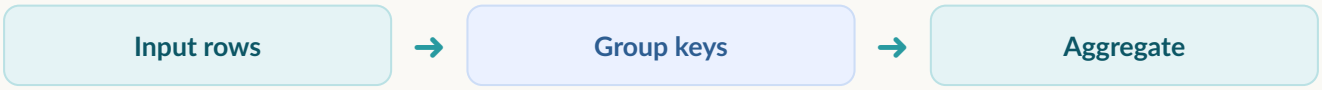
How do WHERE and HAVING differ?

SHORT INTERVIEW ANSWER

WHERE filters input rows; HAVING filters groups after aggregation.

TRACE THE EXAMPLE

```
SELECT department_id, COUNT(*) FROM employees
WHERE salary IS NOT NULL GROUP BY department_id HAVING COUNT(*) >= 2;
```



WHY THIS WORKS

Row predicates reduce eligible inputs, while aggregate predicates belong at group level. Some engines allow HAVING in additional forms, but the distinction remains useful. Do not filter rows needed by another aggregate unintentionally.

WATCH OUT

HAVING is not merely a slower version of WHERE.

FOLLOW-UP QUESTION + ANSWER

What changes if salary filtering is removed?

Removing WHERE salary IS NOT NULL allows the unknown-salary employee into their group, changing counts if included; averages still ignore null salaries.

CHAPTER 06 • AGGREGATION AND GROUPING

QUESTION Q029

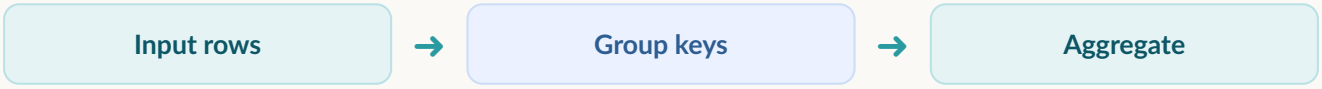
How do you calculate conditional totals?

SHORT INTERVIEW ANSWER

Aggregate CASE expressions or a supported engine-specific filtered aggregate.

TRACE THE EXAMPLE

```
SELECT SUM(CASE WHEN status='PAID' THEN amount ELSE 0 END) AS paid_total
FROM orders;
```



WHY THIS WORKS

CASE makes this query portable across the main engines. SUM over no input rows returns NULL, so use COALESCE when a zero total is the required contract. Keep cancelled/pending amounts separate when the report demands them.

WATCH OUT

Returning zero is a business/reporting choice, not automatic SUM behavior.

FOLLOW-UP QUESTION + ANSWER

How do you compute paid and pending counts together?

SUM(CASE WHEN status='PAID' THEN 1 ELSE 0 END) and SUM(CASE WHEN status='PENDING' THEN 1 ELSE 0 END), together in one SELECT.

CHAPTER 06 • AGGREGATION AND GROUPING

QUESTION Q030

Why can AVG(salary) differ from SUM(salary)/ COUNT(*)?

SHORT INTERVIEW ANSWER

AVG ignores null salary values, while COUNT(*) includes rows without a known salary.

TRACE THE EXAMPLE

```
Known salaries total=410000; known count=5; all rows=6
AVG(salary)=82000; total/all rows is a different measure
```

Input rows → Group keys → Aggregate

WHY THIS WORKS

Use the correct denominator and a decimal arithmetic expression if computing ratios. Some engines perform integer division for integer operands, so cast or use an appropriate numeric literal.

WATCH OUT

Missing salary does not automatically mean zero compensation.

FOLLOW-UP QUESTION + ANSWER

What denominator does your business metric require?

For mean known salary, divide by COUNT(salary); for payroll per headcount, choose a documented rule for unknown salary and use the appropriate population.

CHAPTER 07 • SUBQUERIES, EXISTS AND NULL-SAFE ANTI-JOINS

QUESTION Q031

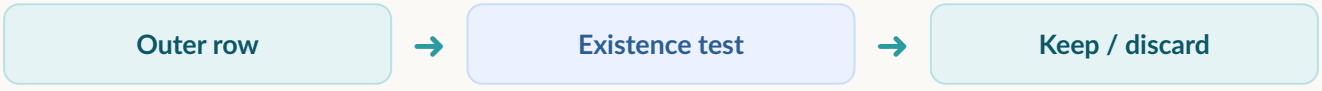
What is a scalar subquery?

SHORT INTERVIEW ANSWER

A scalar subquery supplies one value; no row normally yields NULL, and more than one row is an error under PostgreSQL/MySQL semantics.

TRACE THE EXAMPLE

```
SELECT name FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```



WHY THIS WORKS

An aggregate without grouping supplies one value even for empty input, though AVG may return NULL. A grouped subquery can produce many rows and cannot simply act as one scalar. Keep cardinality explicit.

WATCH OUT

SQLite's scalar-subquery behavior differs for multiple rows.

FOLLOW-UP QUESTION + ANSWER

How would you guarantee one returned value?

Make the subquery aggregate without GROUP BY, or use a unique-key filter. Do not arbitrarily LIMIT 1 unless that is a valid business selection.

CHAPTER 07 • SUBQUERIES, EXISTS AND NULL-SAFE ANTI-JOINS

QUESTION Q032

What is a correlated subquery?

SHORT INTERVIEW ANSWER

It references an outer row, making its logical result depend on that row's values.

TRACE THE EXAMPLE

```
SELECT e.name FROM employees e WHERE e.salary >
 (SELECT AVG(x.salary) FROM employees x WHERE x.department_id=e.department_id);
```

Outer row → Existence test → Keep / discard

WHY THIS WORKS

This finds employees above their department average. The optimizer may transform the query rather than literally execute one independent full scan per row. Verify the plan instead of making a universal performance claim.

WATCH OUT

Correlated does not automatically mean slow.

FOLLOW-UP QUESTION + ANSWER

Can a window aggregate express the same rule?

Yes: AVG(salary) OVER (PARTITION BY department_id) in a CTE, then filter salary > department_avg outside the CTE.

CHAPTER 07 • SUBQUERIES, EXISTS AND NULL-SAFE ANTI-JOINS

QUESTION Q033

When should you use EXISTS?

SHORT INTERVIEW ANSWER

Use EXISTS when the requirement is whether at least one matching row exists, without multiplying the outer rows.

TRACE THE EXAMPLE

```
SELECT c.customer_id FROM customers c WHERE EXISTS
 (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id AND o.status='PAID');
```

Outer row → Existence test → Keep / discard

WHY THIS WORKS

A customer qualifies once even with several paid orders. A join needs DISTINCT or aggregation if the required result is one row per customer. EXISTS does not depend on the selected literal being 1.

WATCH OUT

SELECT 1 is convention, not a special guarantee of faster execution.

FOLLOW-UP QUESTION + ANSWER

How would you find customers with no orders?

SELECT c.customer_id FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id); it naturally includes customers without rows.

CHAPTER 07 • SUBQUERIES, EXISTS AND NULL-SAFE ANTI-JOINS

QUESTION Q034

Why is NOT IN dangerous with NULLs?

SHORT INTERVIEW ANSWER

If the membership set contains NULL, nonmatching comparisons can become unknown and be filtered out.

TRACE THE EXAMPLE

```
SELECT c.customer_id FROM customers c WHERE c.customer_id NOT IN
 (SELECT manager_id FROM employees);
```

Outer row → Existence test → Keep / discard

WHY THIS WORKS

The fixture has NULL manager IDs, so this is a poor exclusion rule. Prefer a logically appropriate NOT EXISTS, or prove/filter non-null values on both sides. Do not replace one construct with another without checking outer null semantics.

WATCH OUT

NOT IN and NOT EXISTS are not universally equivalent.

FOLLOW-UP QUESTION + ANSWER

What if the outer identifier is NULL?

An outer NULL requires explicit meaning. With NOT EXISTS, null-safe correlation is necessary only if NULL is a legitimate identity; usually a NOT NULL key is preferable.

CHAPTER 07 • SUBQUERIES, EXISTS AND NULL-SAFE ANTI-JOINS

QUESTION Q035

How do anti-joins find missing relationships?

SHORT INTERVIEW ANSWER

NOT EXISTS can retain rows for which no qualifying related row is present.

TRACE THE EXAMPLE

```
SELECT c.customer_id FROM customers c WHERE NOT EXISTS
 (SELECT 1 FROM orders o WHERE o.customer_id=c.customer_id);
```

Outer row → Existence test → Keep / discard

WHY THIS WORKS

A left join plus IS NULL on a non-null right-side key can also express absence. Keep additional matching filters in the appropriate place. Use the business definition of missing: no orders differs from no paid orders.

WATCH OUT

Testing a nullable right-side field can misclassify a real match.

FOLLOW-UP QUESTION + ANSWER

Which key should the IS NULL test use?

Check a non-null, guaranteed right-side key such as o.order_id IS NULL after LEFT JOIN; do not use a nullable payload column.

CHAPTER 08 • CTES AND RECURSIVE QUERIES

QUESTION Q036

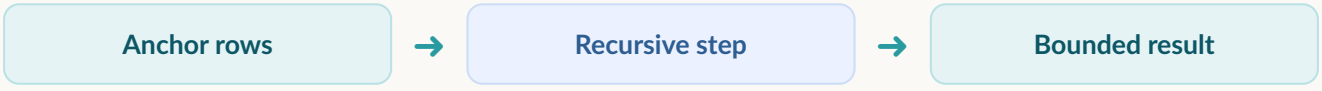
What is a CTE?

SHORT INTERVIEW ANSWER

A common table expression names a query result within one statement using WITH.

TRACE THE EXAMPLE

```
WITH paid AS (SELECT * FROM orders WHERE status='PAID')
SELECT customer_id, SUM(amount) FROM paid GROUP BY customer_id;
```



WHY THIS WORKS

It improves readability and supports reuse within that statement. It is not automatically a stored table or permanently cached result. Optimization/materialization behavior differs by engine, version and query structure.

WATCH OUT

A CTE is not always faster than a subquery.

FOLLOW-UP QUESTION + ANSWER

What would you inspect to see its physical treatment?

Check EXPLAIN or EXPLAIN ANALYZE and the engine/version rules; CTEs may be inlined or materialized depending on plan and syntax.

CHAPTER 08 • CTES AND RECURSIVE QUERIES

QUESTION Q037

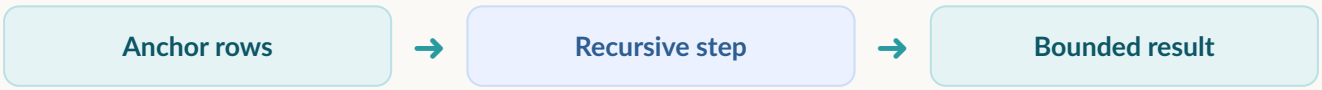
How do a CTE, derived table and temporary table differ?

SHORT INTERVIEW ANSWER

A CTE and derived table are statement-local query constructs; a temporary table is a separate session/transaction-scoped object under engine rules.

TRACE THE EXAMPLE

```
Derived: FROM (SELECT ...) AS q
CTE: WITH q AS (SELECT ...)
Temporary: CREATE TEMPORARY TABLE q (...)
```



WHY THIS WORKS

A temporary table can support explicit staging/indexes and reuse across statements, with creation/cleanup and transaction semantics to review. Choose from reuse, plan control and lifecycle needs rather than style alone.

WATCH OUT

Temporary-table scope and DDL behavior vary by database.

FOLLOW-UP QUESTION + ANSWER

When is materialized staging worth the cost?

Stage results when expensive intermediate data is reused or needs indexing and stable lifetime. Measure creation, cleanup and statistics costs.

CHAPTER 08 • CTES AND RECURSIVE QUERIES

QUESTION Q038

What does a recursive CTE contain?

SHORT INTERVIEW ANSWER

It combines an anchor term with a recursive term that consumes prior results until no new rows are produced.

TRACE THE EXAMPLE

```
WITH RECURSIVE org AS (
 SELECT employee_id, manager_id, 0 AS depth FROM employees WHERE employee_id=1
 UNION ALL
 SELECT e.employee_id,e.manager_id,o.depth+1 FROM employees e
 JOIN org o ON e.manager_id=o.employee_id WHERE o.depth<10)
SELECT * FROM org;
```

Anchor rows → Recursive step → Bounded result

WHY THIS WORKS

This walks Sai's reporting subtree in the fixture. Add a path/cycle policy for general graphs; a depth cap is a bound, not proof the graph is acyclic. Output order needs ORDER BY separately.

WATCH OUT

Recursion does not imply deterministic traversal order.

FOLLOW-UP QUESTION + ANSWER

How would you report a detected cycle?

Carry a visited-ID path or a cycle marker and emit the offending edge; do not confuse a shared descendant with a loop.

CHAPTER 08 • CTES AND RECURSIVE QUERIES

QUESTION Q039

How do you prevent a recursive query running indefinitely?

SHORT INTERVIEW ANSWER

Use valid hierarchy constraints, cycle detection and a reviewed bound; do not depend only on unlimited data being well formed.

TRACE THE EXAMPLE

```
Track visited IDs or use supported cycle syntax
Reject revisiting a node on the current path
```

Anchor rows → Recursive step → Bounded result

WHY THIS WORKS

UNION deduplication may stop some recursions but does not generally replace path-aware cycle logic when rows include changing depth/path values. Engine recursion limits differ. Test cyclic, deep and shared-descendant cases.

WATCH OUT

Incrementing depth makes repeated visits distinct rows.

FOLLOW-UP QUESTION + ANSWER

Is a shared child a cycle?

No. A cycle revisits a node on the current traversal path. Two independent parent paths reaching the same child form a DAG, not necessarily a cycle.

CHAPTER 08 • CTES AND RECURSIVE QUERIES

QUESTION Q040

What is a useful multi-stage CTE interview pattern?

SHORT INTERVIEW ANSWER

Break grain-changing operations into named stages, then join/filter their documented outputs.

TRACE THE EXAMPLE

```
WITH paid_totals AS (
 SELECT customer_id, SUM(amount) AS total FROM orders
 WHERE status='PAID' GROUP BY customer_id)
SELECT c.name,p.total FROM customers c JOIN paid_totals p
 ON p.customer_id=c.customer_id;
```

Anchor rows → Recursive step → Bounded result

WHY THIS WORKS

The first stage creates one row per paying customer. The next joins that grain to customers without detail multiplication. Readable stages make correctness easier to assess, while the optimizer still chooses physical operations.

WATCH OUT

Naming a stage does not enforce uniqueness without the query/constraints.

FOLLOW-UP QUESTION + ANSWER

How would you include zero-paying customers?

Start with customers and LEFT JOIN the per-customer payment aggregate, COALESCE total to zero if required; preserve the empty customer.

CHAPTER 09 • WINDOW FUNCTIONS

QUESTION Q041

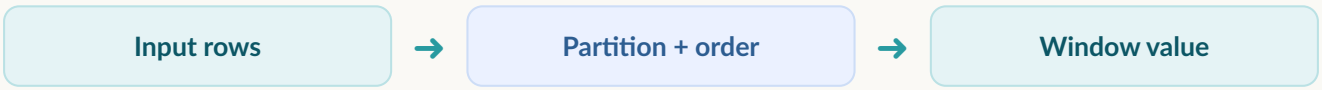
How do window functions differ from GROUP BY?

SHORT INTERVIEW ANSWER

Window functions calculate across related rows without collapsing detail rows into one group row.

TRACE THE EXAMPLE

```
SELECT employee_id, department_id, salary,
 AVG(salary) OVER(PARTITION BY department_id) AS dept_avg
FROM employees;
```



WHY THIS WORKS

Each employee retains an output row with its department average. A partition is a calculation grouping, not table storage partitioning. Window ordering and final result ordering are separate.

WATCH OUT

OVER(ORDER BY ...) does not guarantee final display order.

FOLLOW-UP QUESTION + ANSWER

What is the output grain here?

The output remains one row per base order because the window annotates rows; GROUP BY would collapse rows into groups.

CHAPTER 09 • WINDOW FUNCTIONS

QUESTION Q042

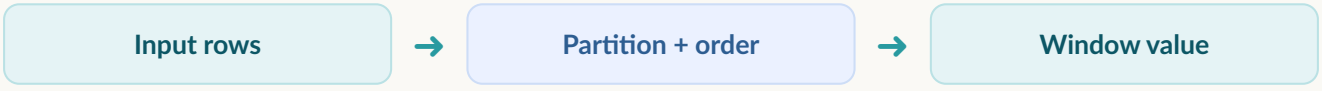
How do ROW_NUMBER, RANK and DENSE_RANK differ?

SHORT INTERVIEW ANSWER

ROW_NUMBER numbers rows; RANK gives peers the same rank with gaps; DENSE_RANK gives peers the same rank without gaps.

TRACE THE EXAMPLE

```
Salaries 100000,80000,80000,60000
ROW_NUMBER: 1,2,3,4; RANK: 1,2,2,4; DENSE_RANK: 1,2,2,3
```



WHY THIS WORKS

Peer equality follows the window ORDER BY expressions. Adding employee_id to a salary-ranking order breaks salary ties. Use a separate deterministic tie-breaker for row-number selection when exactly N rows are required.

WATCH OUT

Top two rows and top two distinct salaries are different requirements.

FOLLOW-UP QUESTION + ANSWER

Which function fits second distinct salary?

DENSE_RANK on salary DESC finds the second distinct level including ties. Filter rank=2 in an outer query.

CHAPTER 09 • WINDOW FUNCTIONS

QUESTION Q043

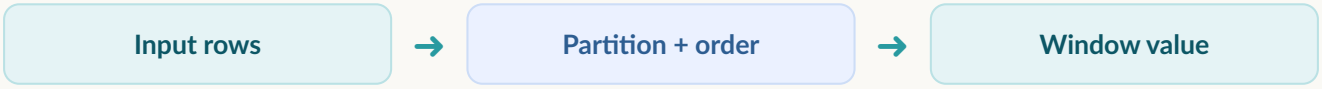
Why specify an explicit running-total frame?

SHORT INTERVIEW ANSWER

ROWS with a unique order makes the intended per-row accumulation explicit instead of relying on a default peer-sensitive frame.

TRACE THE EXAMPLE

```
SUM(amount) OVER (ORDER BY ordered_at, order_id
 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```



WHY THIS WORKS

With ordered aggregate windows, default frames can include all peers at the current ordering value. An order_id tie-breaker plus ROWS makes row progression clear. Partition when each customer needs a separate total.

WATCH OUT

Same-date rows can share a total under a peer-based frame.

FOLLOW-UP QUESTION + ANSWER

What changes if order_id is removed?

Without order_id, equal-date rows become peers; RANGE or default frame behavior may produce jumps. Specify ROWS and a unique tie-breaker for deterministic progression.

CHAPTER 09 • WINDOW FUNCTIONS

QUESTION Q044

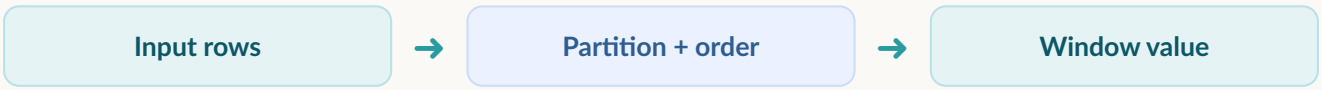
How do LAG and LEAD help?

SHORT INTERVIEW ANSWER

They access values from earlier/later rows in the window's logical order without a self join.

TRACE THE EXAMPLE

```
SELECT order_id, amount,
 LAG(amount) OVER(PARTITION BY customer_id ORDER BY ordered_at,order_id) AS prev
FROM orders;
```



WHY THIS WORKS

The first row has no preceding value unless a default is supplied. Distinguish a missing preceding row from a preceding row whose value is NULL. Add deterministic ordering and decide whether cancelled rows belong in the sequence.

WATCH OUT

Offset means row position, not elapsed days.

FOLLOW-UP QUESTION + ANSWER

How would you calculate a change from the previous paid order?

Use LAG(amount) OVER (PARTITION BY customer_id ORDER BY ordered_at, order_id) on paid orders, then subtract the prior amount.

CHAPTER 09 • WINDOW FUNCTIONS

QUESTION Q045

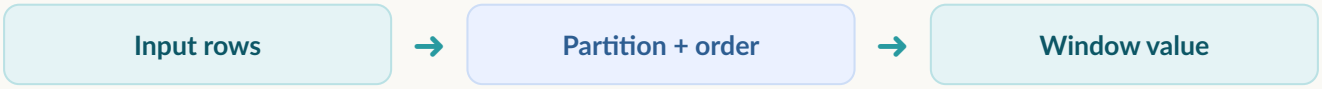
How do you filter a window result in PostgreSQL/ MySQL?

SHORT INTERVIEW ANSWER

Compute it in a CTE/derived table, then filter in an outer query.

TRACE THE EXAMPLE

```
WITH ranked AS (
 SELECT employee_id,salary,DENSE_RANK() OVER(ORDER BY salary DESC) AS r
 FROM employees WHERE salary IS NOT NULL)
SELECT employee_id,salary FROM ranked WHERE r=2;
```



WHY THIS WORKS

WHERE in the same query level runs before the window result is available. QUALIFY exists in some other engines but is not part of these baseline examples. Keep final ORDER BY explicit.

WATCH OUT

A window alias in same-level WHERE is not portable SQL.

FOLLOW-UP QUESTION + ANSWER

How do you return exactly two rows instead of ties?

Use ROW_NUMBER with unique ordering and filter row_num<=2, instead of DENSE_RANK which may include extra tied rows.

CHAPTER 10 • COMMON INTERVIEW QUERY CHALLENGES

QUESTION Q046

How do you find the second-highest distinct salary?

SHORT INTERVIEW ANSWER

Rank non-null distinct salary levels, or take the maximum salary below the global maximum.

TRACE THE EXAMPLE

```
SELECT MAX(salary) AS second_salary FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Rank / ties → Select rule → Expected rows

WHY THIS WORKS

This returns 90000 in the fixture and NULL if fewer than two distinct known salaries exist. It returns a salary, not all employees earning it. Use DENSE_RANK when employee rows at a selected level are required.

WATCH OUT

OFFSET 1 on raw employee rows can select another tied maximum.

FOLLOW-UP QUESTION + ANSWER

How do you return all second-level employees?

Compute the second distinct amount in a CTE and filter employees whose salary equals it; tied employees are all included.

CHAPTER 10 • COMMON INTERVIEW QUERY CHALLENGES

QUESTION Q047

How do you select top N salaries per department?

SHORT INTERVIEW ANSWER

Use DENSE_RANK for salary levels or ROW_NUMBER for an exact row count, with an explicit tie policy.

TRACE THE EXAMPLE

```
WITH r AS (
 SELECT employee_id,department_id,salary,
 DENSE_RANK() OVER(PARTITION BY department_id ORDER BY salary DESC) AS pos
 FROM employees WHERE salary IS NOT NULL)
SELECT * FROM r WHERE pos<=2;
```

Rank / ties → Select rule → Expected rows

WHY THIS WORKS

Engineering returns three employees because two share its second level. Exclude or handle unassigned/null values as required. For exactly two people, ROW_NUMBER with salary then employee_id gives deterministic selection.

WATCH OUT

Adding a unique ID to DENSE_RANK changes the salary-level definition.

FOLLOW-UP QUESTION + ANSWER

How many rows should Engineering return?

Engineering has three people across two distinct salary levels: one at 100,000 and two tied at 80,000.

CHAPTER 10 • COMMON INTERVIEW QUERY CHALLENGES

QUESTION Q048

How do you find duplicate business values?

SHORT INTERVIEW ANSWER

Group by the business identity columns and retain groups with more than one row.

TRACE THE EXAMPLE

```
SELECT email,COUNT(*) FROM contacts GROUP BY email HAVING COUNT(*)>1;
```

Rank / ties → Select rule → Expected rows

WHY THIS WORKS

Define case/space normalization and NULL policy before calling values duplicates. For removal, first choose a deterministic retained row and review references. Duplicates by email are not necessarily duplicates by the entire row.

WATCH OUT

DISTINCT only hides duplicates in one result; it does not repair stored data.

FOLLOW-UP QUESTION + ANSWER

Which row should survive a cleanup?

Choose a survivor by a documented rule such as earliest verified row, then use ROW_NUMBER partitioned by the normalized business key, ordered by chosen priority and ID.

CHAPTER 10 • COMMON INTERVIEW QUERY CHALLENGES

QUESTION Q049

How do you solve gaps-and-islands questions?

SHORT INTERVIEW ANSWER

Derive a stable group key or change marker for consecutive runs, then aggregate by that group.

TRACE THE EXAMPLE

```
Consecutive integer values: value - ROW_NUMBER() OVER(ORDER BY value)
Equal differences identify one island after deduplication
```

Rank / ties → Select rule → Expected rows

WHY THIS WORKS

Define consecutive precisely: integer +1, adjacent dates or adjacent events. Deduplicate input when necessary, handle partitions and use dialect-specific date arithmetic carefully. A LAG-based start marker is another useful method.

WATCH OUT

Missing weekends are not a gap if the business uses working days.

FOLLOW-UP QUESTION + ANSWER

What interval defines adjacency?

Define whether adjacent means consecutive integers, dates, or gaps no larger than a threshold; derive groups using that specific definition.

CHAPTER 10 • COMMON INTERVIEW QUERY CHALLENGES

QUESTION Q050

How do you find customers who bought every required product?

SHORT INTERVIEW ANSWER

Use relational division, often expressed as no required product lacking a corresponding purchase.

TRACE THE EXAMPLE

```
SELECT c.customer_id FROM customers c WHERE NOT EXISTS (
 SELECT 1 FROM required_products p WHERE NOT EXISTS (
  SELECT 1 FROM purchases x WHERE x.customer_id=c.customer_id
  AND x.product_id=p.product_id));
```

Rank / ties → Select rule → Expected rows

WHY THIS WORKS

The two NOT EXISTS clauses express universal coverage. Decide whether duplicate purchases, cancelled orders and an empty required-product set count. Under this formulation, an empty required set qualifies every customer.

WATCH OUT

Counting raw purchase rows can overcount duplicates.

FOLLOW-UP QUESTION + ANSWER

What should the empty-set case mean for the business?

Mathematically every customer qualifies when the required set is empty, but the product may choose no result; state the intended business rule explicitly.

CHAPTER 11 • DATA CHANGES, UPSERT AND DEDUPLICATION

QUESTION Q051

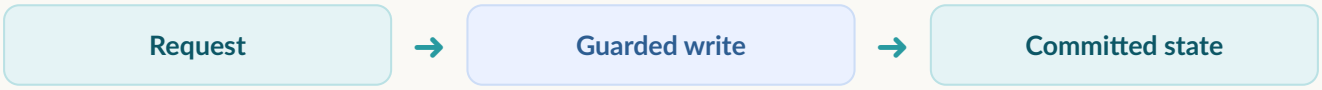
How do INSERT, UPDATE and DELETE target data?

SHORT INTERVIEW ANSWER

INSERT adds rows; UPDATE modifies qualifying rows; DELETE removes qualifying rows.

TRACE THE EXAMPLE

```
UPDATE employees SET salary=salary+1000 WHERE employee_id=5;
```



WHY THIS WORKS

An omitted or incorrect predicate can affect many rows. For practice, use disposable fixtures and inspect the same predicate with SELECT. Production changes need reviewed transactions, scope and affected-row evidence.

WATCH OUT

A precheck SELECT does not prevent concurrent changes.

FOLLOW-UP QUESTION + ANSWER

Which invariant belongs in the write predicate?

Put the stock or version check in UPDATE ... WHERE, and verify affected rows. Constraints still enforce the final invariant.

CHAPTER 11 • DATA CHANGES, UPSERT AND DEDUPLICATION

QUESTION Q052

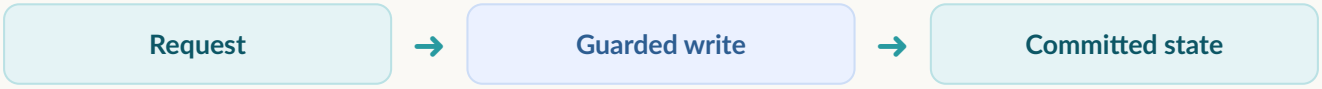
What is an upsert?

SHORT INTERVIEW ANSWER

It inserts or updates according to a conflict rule, with engine-specific syntax and semantics.

TRACE THE EXAMPLE

```
-- PostgreSQL: INSERT ... ON CONFLICT (...) DO UPDATE ...
-- MySQL: INSERT ... ON DUPLICATE KEY UPDATE ...
```



WHY THIS WORKS

A unique constraint defines conflict identity. Concurrency, affected-row counts and chosen update expressions matter. Do not treat upsert as a universal replacement for domain state-transition checks.

WATCH OUT

PostgreSQL conflict-target behavior differs from MySQL unique-key conflict handling.

FOLLOW-UP QUESTION + ANSWER

Which columns define the conflict?

Use the business's unique identifier, for example (tenant_id, external_payment_id), rather than choosing any convenient column.

CHAPTER 11 • DATA CHANGES, UPSERT AND DEDUPLICATION

QUESTION Q053

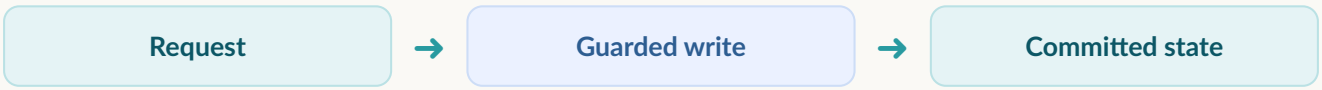
Why is SELECT-then-INSERT unsafe for uniqueness?

SHORT INTERVIEW ANSWER

Two transactions can both observe absence and attempt insertion; the database constraint must arbitrate concurrent writers.

TRACE THE EXAMPLE

```
A: no booking -> insert
B: no booking -> insert
UNIQUE(slot_id) decides whether both can exist
```



WHY THIS WORKS

Handle the conflict as a documented outcome or retry where appropriate. Locking one nonexistent row is not a universal solution. For a request ID, persist the result/claim under an appropriate transaction contract.

WATCH OUT

Application prechecks are not a substitute for UNIQUE.

FOLLOW-UP QUESTION + ANSWER

How do you distinguish duplicate request from conflicting booking?

Use a unique idempotency/slot key; on conflict read the stored request hash and return its prior result only if the new request matches.

CHAPTER 11 • DATA CHANGES, UPSERT AND DEDUPLICATION

QUESTION Q054

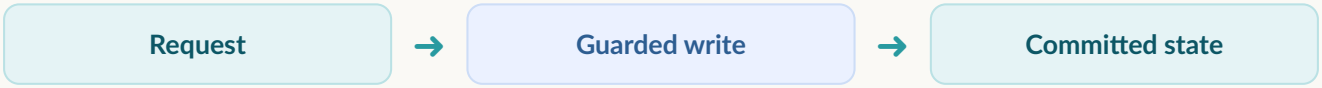
How do you remove duplicates deterministically?

SHORT INTERVIEW ANSWER

Identify candidates using ROW_NUMBER over the business identity, retain one chosen key, then perform an engine-specific reviewed delete.

TRACE THE EXAMPLE

```
SELECT contact_id,email,
 ROW_NUMBER() OVER(PARTITION BY email ORDER BY contact_id) AS n
FROM contacts;
-- n>1 are candidates, not automatically safe to delete.
```



WHY THIS WORKS

References, merged fields and audit requirements determine whether deletion is valid. Use a migration workflow and enforce future uniqueness after cleanup. MySQL/ PostgreSQL delete-from-CTE syntax differs.

WATCH OUT

The minimum ID is not necessarily the best business record.

FOLLOW-UP QUESTION + ANSWER

What if another table references the row being removed?

Update foreign-key references to the chosen survivor inside a safe transaction before deleting duplicate rows; verify referential integrity.

CHAPTER 11 • DATA CHANGES, UPSERT AND DEDUPLICATION

QUESTION Q055

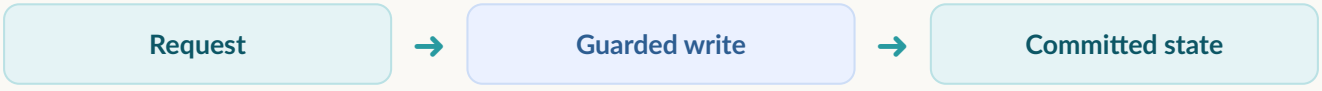
How do you prevent lost updates in a write?

SHORT INTERVIEW ANSWER

Use a resource-level atomic expression or an expected-version predicate instead of blindly writing a stale value.

TRACE THE EXAMPLE

```
UPDATE inventory SET available=available-1
WHERE item_id=:id AND available>0;
```



WHY THIS WORKS

Inspect affected rows: one means the guarded mutation succeeded, zero means unavailable/missing under the contract. A broader workflow may still require a transaction and stable request identity. Version checks fit replacing complex stale state.

WATCH OUT

Atomic stock decrement alone does not deduplicate repeated requests.

FOLLOW-UP QUESTION + ANSWER

How would you handle a timeout after the update committed?

Use a stable request ID, check transaction outcome and read current version/state before retrying. A timeout does not prove rollback.

CHAPTER 12 • NORMALIZATION AND SCHEMA DESIGN

QUESTION Q056

What is normalization and why use it?

SHORT INTERVIEW ANSWER

Normalization organizes facts according to dependencies to reduce update, insert and delete anomalies.

TRACE THE EXAMPLE

```
Store department name once in departments
employees reference department_id
```

Entity keys → Dependencies → Normalized tables

WHY THIS WORKS

Duplicating department names across employee rows can create conflicting copies. Normalization improves representation, while constraints and transaction logic enforce actual integrity. Query needs may justify deliberate derived read models.

WATCH OUT

More tables do not automatically mean a better model.

FOLLOW-UP QUESTION + ANSWER

What update anomaly does the split remove?

The split stops one customer's detail being repeated in many independent rows, so updating a customer name no longer requires editing each order.

CHAPTER 12 • NORMALIZATION AND SCHEMA DESIGN

QUESTION Q057

What do 1NF, 2NF and 3NF mean?

SHORT INTERVIEW ANSWER

In the common teaching model, 1NF avoids repeating groups, 2NF removes partial-key dependencies, and 3NF removes inappropriate transitive dependencies of non-key facts.

TRACE THE EXAMPLE

```
Enrollment(student_id,course_id,student_name,grade)
student_name depends on student_id, not the whole composite key
```

Entity keys → Dependencies → Normalized tables

WHY THIS WORKS

Treat 2NF/3NF definitions through candidate keys and functional dependencies, not merely the presence of foreign keys. Atomicity of a value depends on the modeled domain. Decompose carefully and preserve correct joins/constraints.

WATCH OUT

A single-column surrogate key does not cure all business dependency anomalies.

FOLLOW-UP QUESTION + ANSWER

Which relation should own student_name?

student_name belongs to Students keyed by student_id, not repeated on each enrollment relationship.

CHAPTER 12 • NORMALIZATION AND SCHEMA DESIGN

QUESTION Q058

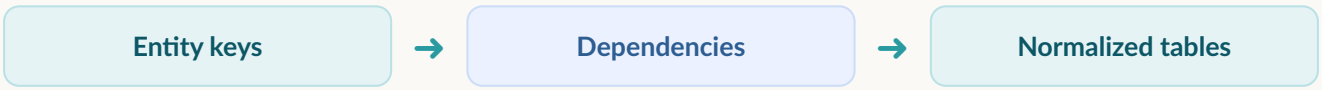
How does BCNF differ from 3NF?

SHORT INTERVIEW ANSWER

BCNF requires every nontrivial functional dependency's determinant to be a superkey; 3NF permits certain dependencies involving prime attributes.

TRACE THE EXAMPLE

```
R(student,course,instructor)
FDs: (student,course) -> instructor; instructor -> course
Candidate keys: (student,course), (student,instructor)
```



WHY THIS WORKS

Assume each instructor teaches one course, and each student has one instructor for a course. Instructor determines course but is not a superkey, so the relation violates BCNF. Course is a prime attribute (part of a candidate key), so this dependency is permitted by 3NF. Decomposing into InstructorCourse and StudentInstructor can lose direct preservation of the student/course instructor rule. BCNF can remove anomalies remaining in a 3NF design, but dependency preservation can be a tradeoff. Interview answers should state actual dependencies for an example rather than label a table from its column count.

WATCH OUT

Every determinant is not automatically a primary key.

FOLLOW-UP QUESTION + ANSWER

What is a prime attribute?

A prime attribute is part of some candidate key. BCNF requires every determinant to be a superkey; 3NF allows some dependencies whose dependent attribute is prime.

CHAPTER 12 • NORMALIZATION AND SCHEMA DESIGN

QUESTION Q059

How do you model many-to-many relationships?

SHORT INTERVIEW ANSWER

Use an association table with references and an appropriate uniqueness identity.

TRACE THE EXAMPLE

```
CREATE TABLE enrollment (
 student_id INTEGER, course_id INTEGER,
 PRIMARY KEY(student_id,course_id)
);
```

Entity keys → Dependencies → Normalized tables

WHY THIS WORKS

Add foreign keys to the relevant tables in a real schema and store relationship attributes such as grade on the association. Some domains allow repeated enrollment attempts, requiring another identity dimension.

WATCH OUT

A comma-separated list of course IDs weakens relational integrity/querying.

FOLLOW-UP QUESTION + ANSWER

How would multiple attempts change the key?

Use an enrollment/attempt identity, e.g. PRIMARY KEY(student_id, course_id, attempt_number) or a unique attempt_id plus a documented composite uniqueness rule.

CHAPTER 12 • NORMALIZATION AND SCHEMA DESIGN

QUESTION Q060

When is denormalization reasonable?

SHORT INTERVIEW ANSWER

Use it for measured read needs with an explicit source, refresh/invalidation policy and reconciliation plan.

TRACE THE EXAMPLE

```
Reporting projection stores paid_total per customer
Source orders remain authoritative
```

Entity keys → Dependencies → Normalized tables

WHY THIS WORKS

Duplicated/derived facts trade write complexity and freshness for read speed. Materialized summaries need correct handling of updates, deletes and late events. Establish whether an inconsistent read is acceptable.

WATCH OUT

Storing a total does not ensure it stays correct.

FOLLOW-UP QUESTION + ANSWER

How do you rebuild or verify the projection?

Rebuild from authoritative events/rows and compare row counts, totals, checksums or sampled business invariants before switching readers.

CHAPTER 13 • INDEXES AND ACCESS-PATH DESIGN

QUESTION Q061

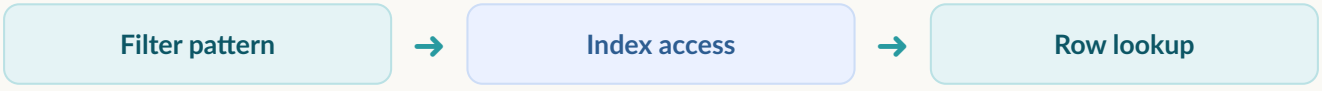
What does an index provide?

SHORT INTERVIEW ANSWER

It supplies an alternative access path that can reduce lookup, filtering or ordering work at storage and write-maintenance cost.

TRACE THE EXAMPLE

```
CREATE INDEX ix_orders_customer_date
ON orders(customer_id,ordered_at,order_id);
```



WHY THIS WORKS

An index is not guaranteed to be selected. Selectivity, statistics, row width and ordering needs affect cost. Additional indexes consume space and make inserts/updates/deletes more expensive.

WATCH OUT

More indexes do not monotonically improve performance.

FOLLOW-UP QUESTION + ANSWER

Which workload justifies this index?

A measured query repeatedly filters by customer_id and ordered_at, so an index beginning with customer_id then date may be useful. Verify with the plan.

CHAPTER 13 • INDEXES AND ACCESS-PATH DESIGN

QUESTION Q062

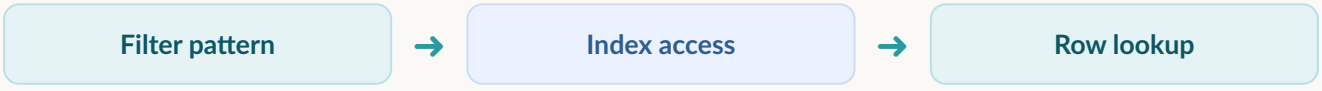
How does composite-index column order matter?

SHORT INTERVIEW ANSWER

The ordered key structure makes some leading-column predicates and ordering combinations more useful than others.

TRACE THE EXAMPLE

```
(customer_id,ordered_at,order_id)
WHERE customer_id=? ORDER BY ordered_at,order_id
```



WHY THIS WORKS

Equality, range and requested ordering should guide the design. Leftmost-prefix rules are a useful baseline, but optimizer/index features can offer exceptions or alternative scans. Measure rather than declaring later columns never usable.

WATCH OUT

An index on (a,b) is not generally equivalent to (b,a).

FOLLOW-UP QUESTION + ANSWER

What happens with only ordered_at in the predicate?

A composite index beginning with customer_id may not efficiently locate only ordered_at; test a separate date-leading index based on actual workload.

CHAPTER 13 • INDEXES AND ACCESS-PATH DESIGN

QUESTION Q063

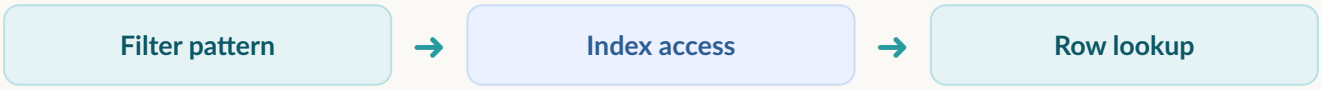
What is a covering or index-only query?

SHORT INTERVIEW ANSWER

The access path may supply required columns from an index, reducing table/heap visits, subject to engine visibility and storage rules.

TRACE THE EXAMPLE

```
Query projects columns present in a chosen index
```



WHY THIS WORKS

PostgreSQL index-only scans still depend on visibility information and can require heap checks. MySQL InnoDB secondary indexes include primary-key information. Include/payload syntax and index size tradeoffs differ.

WATCH OUT

Covering does not mean zero I/O in every engine.

FOLLOW-UP QUESTION + ANSWER

How would you verify actual heap/table reads?

Use EXPLAIN (ANALYZE, BUFFERS) in PostgreSQL and check heap fetches; in MySQL compare index coverage and actual row reads using execution evidence.

CHAPTER 13 • INDEXES AND ACCESS-PATH DESIGN

QUESTION Q064

What makes a predicate sargable?

SHORT INTERVIEW ANSWER

It can be matched effectively to an available indexed access path, depending on types, expressions and engine capabilities.

TRACE THE EXAMPLE

```
Better baseline: ordered_at>=:start AND ordered_at<:end
Potential issue: DATE(ordered_at)=:day without matching expression index
```

Filter pattern → Index access → Row lookup

WHY THIS WORKS

Functions, implicit casts, leading wildcards and mismatched collation can affect access. Expression/generated-column indexes can support some transformed predicates. Explain the actual plan, not a universal ban on functions.

WATCH OUT

WHERE syntax alone does not prove an index seek.

FOLLOW-UP QUESTION + ANSWER

How would you support a case-insensitive search?

Use an appropriate normalized/search column with an index, or an engine-supported expression/function index; compare collation behavior.

CHAPTER 13 • INDEXES AND ACCESS-PATH DESIGN

QUESTION Q065

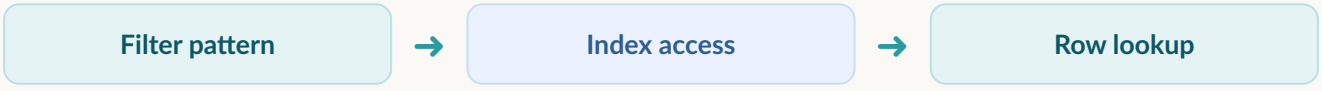
How do clustered storage and secondary indexes differ across engines?

SHORT INTERVIEW ANSWER

MySQL InnoDB stores rows in its clustered primary-key organization; PostgreSQL commonly uses a heap with separate indexes.

TRACE THE EXAMPLE

```
InnoDB secondary entry includes primary-key columns
PostgreSQL secondary index points to heap tuples
```



WHY THIS WORKS

PostgreSQL CLUSTER can physically reorder a table but does not continuously maintain InnoDB-like clustering. A wide InnoDB primary key increases secondary-index storage. These details affect design and lookup cost.

WATCH OUT

Saying every table has the same kind of clustered index is incorrect.

FOLLOW-UP QUESTION + ANSWER

Why might a compact primary key matter?

InnoDB secondary indexes store primary-key values as row locators; a large primary key increases secondary index size and lookup work.

CHAPTER 14 • EXECUTION PLANS AND QUERY TUNING

QUESTION Q066

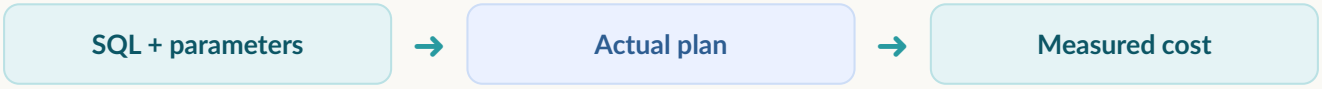
What does EXPLAIN show?

SHORT INTERVIEW ANSWER

It shows the chosen plan and estimates; execution-aware variants can add actual timing and row counts.

TRACE THE EXAMPLE

```
-- PostgreSQL example:
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id=1;
```



WHY THIS WORKS

Analyze forms execute the statement, so do not casually run them on mutating production SQL. Interpret scan, join, filter, sort and aggregate nodes. Engine output and timing semantics differ.

WATCH OUT

Estimated cost is not a duration in milliseconds.

FOLLOW-UP QUESTION + ANSWER

Where would you look for unexpected row multiplication?

Find where actual rows exceed estimates after a join; unexpected many-to-many multiplication often begins at that operator.

CHAPTER 14 • EXECUTION PLANS AND QUERY TUNING

QUESTION Q067

Why can estimated and actual row counts differ?

SHORT INTERVIEW ANSWER

Statistics, data skew, correlations and parameter distributions can make cardinality estimates inaccurate.

TRACE THE EXAMPLE

```
Estimated 10 rows; actual 100000 -> downstream plan may be poor
```

SQL + parameters → Actual plan → Measured cost

WHY THIS WORKS

Refresh appropriate statistics and investigate distribution/correlation rather than only forcing an index. Small sample fixtures cannot reproduce every production estimate. Include representative parameter values.

WATCH OUT

Statistics can be current and still miss important correlations.

FOLLOW-UP QUESTION + ANSWER

Which columns are correlated in this workload?

Correlated tenant_id, status and ordered_at values can mislead independent estimates; inspect per-tenant distribution and supported statistics.

CHAPTER 14 • EXECUTION PLANS AND QUERY TUNING

QUESTION Q068

What are nested-loop, hash and merge joins?

SHORT INTERVIEW ANSWER

They are physical join strategies using repeated lookups, hashing or ordered matching, chosen according to supported operators and costs.

TRACE THE EXAMPLE

```
Small outer + indexed inner -> nested loop can be good
Large equijoin -> hash may fit, subject to memory/support
```

SQL + parameters → Actual plan → Measured cost

WHY THIS WORKS

No algorithm is always best. A nested loop can be disastrous with a huge outer result and expensive inner scans, while efficient indexed lookups can make it ideal. Plan names and availability differ by engine.

WATCH OUT

Rewriting SQL text does not guarantee a specific physical join.

FOLLOW-UP QUESTION + ANSWER

How would a row-estimate error change the decision?

An underestimated outer input may lead to nested-loop work that is much larger than expected; a correct estimate might favor hash/merge join or a different access path.

CHAPTER 14 • EXECUTION PLANS AND QUERY TUNING

QUESTION Q069

Why can a sequential/table scan be the best plan?

SHORT INTERVIEW ANSWER

When much of a table is needed or the table is small, scanning can cost less than many scattered index/table lookups.

TRACE THE EXAMPLE

```
SELECT most columns WHERE status matches 90% of rows
```

SQL + parameters → Actual plan → Measured cost

WHY THIS WORKS

Evaluate total work and latency rather than treating every scan as a defect. Storage/cache state and column width matter. A tiny practice database often favors scans even with an appropriate production index.

WATCH OUT

Index presence is not proof the optimizer should use it.

FOLLOW-UP QUESTION + ANSWER

What measurement would justify a different access path?

Measure elapsed time, buffers/I/O, row counts and selection ratio on representative data. An index helps only when savings exceed lookup cost.

CHAPTER 14 • EXECUTION PLANS AND QUERY TUNING

QUESTION Q070

How do you approach a slow SQL query systematically?

SHORT INTERVIEW ANSWER

Verify semantics and workload, inspect actual plan/rows/waits, then change the dominant cause and compare results.

TRACE THE EXAMPLE

```
Evidence: parameters, row counts, plan, lock waits,
I/O, spills, latency distribution and before/after outputs
```

SQL + parameters → Actual plan → Measured cost

WHY THIS WORKS

Fix incorrect joins first, then access paths/statistics and resource constraints. Changes should preserve results on edge cases and representative data. Measure end-to-end impact, including application fetch and network cost.

WATCH OUT

Optimizing an incorrect query preserves the wrong answer faster.

FOLLOW-UP QUESTION + ANSWER

How would you validate equivalence after rewriting?

Check that the rewrite yields the same columns, row identities and counts for empty, NULL, tie and multi-child cases, preferably with EXCEPT comparisons.

CHAPTER 15 • TRANSACTIONS AND ACID

QUESTION Q071

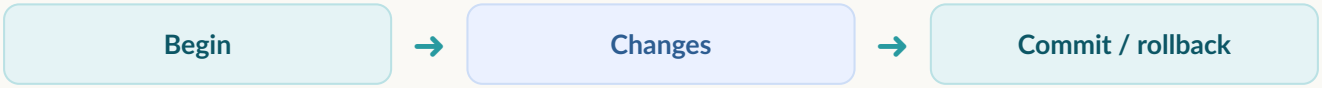
What is an SQL transaction?

SHORT INTERVIEW ANSWER

It groups supported database work into a commit/rollback boundary with configured isolation and engine behavior.

TRACE THE EXAMPLE

```
BEGIN;
UPDATE accounts SET balance=balance-10 WHERE account_id=1;
UPDATE accounts SET balance=balance+10 WHERE account_id=2;
COMMIT;
```



WHY THIS WORKS

The fragment assumes valid rows and business checks within a reviewed transaction. In MySQL, use a transactional engine such as InnoDB and account for autocommit/implicit commits. A single connection owns the local transaction.

WATCH OUT

Two separate autocommitted updates are not one transfer.

FOLLOW-UP QUESTION + ANSWER

What if the destination row is absent?

Abort the unit of work; do not debit if the destination is absent. A foreign key and guarded updates help prevent partial transfer.

CHAPTER 15 • TRANSACTIONS AND ACID

QUESTION Q072

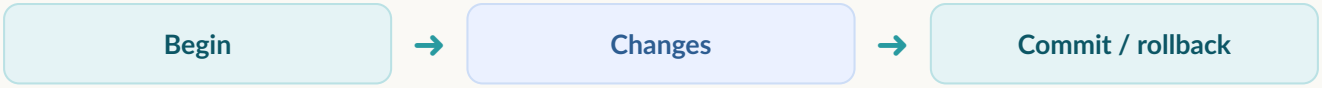
What does ACID mean?

SHORT INTERVIEW ANSWER

Atomicity is a database unit-of-work boundary; consistency preserves enforced invariants; isolation governs concurrent interaction; durability preserves committed work under configured guarantees.

TRACE THE EXAMPLE

```
Atomic rollback != automatic business correctness
Isolation level != universally serial execution
```



WHY THIS WORKS

Application rules and constraints must define a valid transfer or order. Durability depends on database/storage configuration and acknowledged commit boundaries. Avoid saying ACID automatically makes external calls transactional.

WATCH OUT

Consistency is not proof the database inferred every business rule.

FOLLOW-UP QUESTION + ANSWER

Which invariant is actually enforced?

State a concrete invariant, e.g. all committed ledger postings for a transfer balance to zero; enforce it using constraints plus one transaction.

CHAPTER 15 • TRANSACTIONS AND ACID

QUESTION Q073

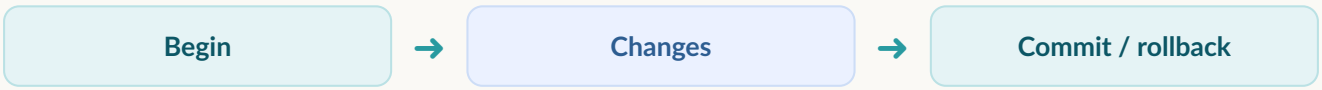
How do COMMIT, ROLLBACK and SAVEPOINT differ?

SHORT INTERVIEW ANSWER

Commit finalizes the transaction; rollback abandons its supported changes; a savepoint can allow partial rollback within it.

TRACE THE EXAMPLE

```
SAVEPOINT before_optional;
-- optional work
ROLLBACK TO SAVEPOINT before_optional;
```



WHY THIS WORKS

Savepoints do not commit the earlier work and are not autonomous nested transactions. Error behavior differs: PostgreSQL commonly leaves a failed transaction needing rollback/recovery, while MySQL statement/transaction effects vary by error.

WATCH OUT

Catching an SQL exception does not guarantee the connection is ready to continue.

FOLLOW-UP QUESTION + ANSWER

What would the transaction manager do after failure?

Roll back failed work (or to a savepoint when safe), then mark the transaction rollback-only under the framework's rules and return an accurate outcome.

CHAPTER 15 • TRANSACTIONS AND ACID

QUESTION Q074

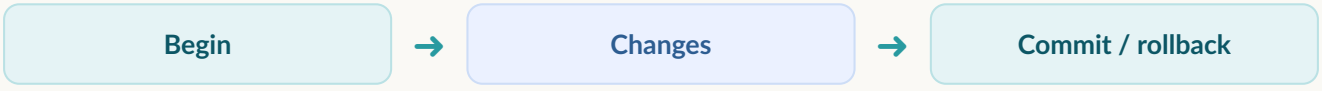
Why keep transactions short?

SHORT INTERVIEW ANSWER

Long transactions can hold locks/resources and retain old visibility/version state, increasing contention and maintenance cost.

TRACE THE EXAMPLE

```
Read required data -> mutate -> commit
Avoid slow HTTP call while holding important row locks
```



WHY THIS WORKS

A correct workflow may need an external call, but design it with explicit intent/outcome handling instead of unlimited lock time. Deadlines and connection-pool capacity matter. PostgreSQL vacuum and InnoDB history cleanup are affected by long-lived snapshots in different ways.

WATCH OUT

Short does not mean splitting an invariant across unsafe commits.

FOLLOW-UP QUESTION + ANSWER

Which work can move outside the transaction?

Move external HTTP calls, reporting and noncritical messages outside the lock-holding transaction; publish committed intent via an outbox when needed.

CHAPTER 15 • TRANSACTIONS AND ACID

QUESTION Q075

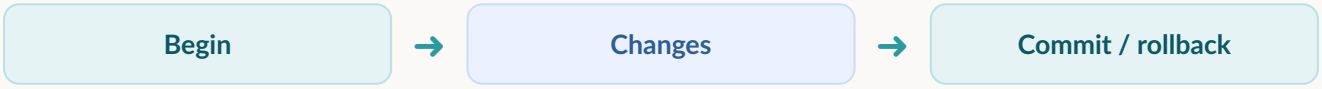
Does a database transaction atomically include Kafka or HTTP?

SHORT INTERVIEW ANSWER

Not by itself. External effects have separate commit/outcome boundaries unless a specific supported distributed protocol is used.

TRACE THE EXAMPLE

```
DB commit succeeds -> event publish fails
Transactional outbox records publication intent with DB work
```



WHY THIS WORKS

An outbox publisher can resend, so downstream deduplication remains required. For payments, use stable provider idempotency and reconciliation of unknown outcomes. Spring transaction annotations do not create universal cross-system atomicity.

WATCH OUT

A method-level transaction cannot roll back an already sent email.

FOLLOW-UP QUESTION + ANSWER

How do you recover a confirmed but unrecorded payment?

Reconcile by the provider idempotency key and persisted intent. If the charge succeeded, record the confirmation without charging again.

CHAPTER 16 • ISOLATION AND MVCC

QUESTION Q076

What anomalies do isolation levels address?

SHORT INTERVIEW ANSWER

Common phenomena include dirty reads, nonrepeatable reads, phantoms and broader serialization anomalies such as write skew.

TRACE THE EXAMPLE

```
Nonrepeatable read: same row value changes between reads
Phantom: a predicate query returns a changed matching row set
```



WHY THIS WORKS

The SQL level names set expectations, but implementations can provide stronger or different details. Snapshot reads and locking reads may behave differently within one engine. Describe a concrete two-session schedule.

WATCH OUT

Repeatable reads of one row do not alone prove a cross-row invariant.

FOLLOW-UP QUESTION + ANSWER

What does write skew look like?

Write skew: two doctors each see the other on call, both update their own on-call flag, and both commit, leaving none available unless protected.

CHAPTER 16 • ISOLATION AND MVCC

QUESTION Q077

How does PostgreSQL Read Committed behave?

SHORT INTERVIEW ANSWER

Each statement normally sees a snapshot of committed data as that statement begins; later statements can see newer commits.

TRACE THE EXAMPLE

```
T1 SELECT -> salary 80000
T2 UPDATE + COMMIT
T1 next SELECT -> may see updated salary
```

Snapshot → Concurrent write → Visible result

WHY THIS WORKS

PostgreSQL treats Read Uncommitted as Read Committed. Locking/data-changing commands have additional rules for concurrent target-row changes. Do not extrapolate ordinary SELECT snapshot behavior to every operation.

WATCH OUT

Transaction begin does not freeze one snapshot at Read Committed.

FOLLOW-UP QUESTION + ANSWER

What changes at Repeatable Read?

In PostgreSQL Repeatable Read, ordinary reads use a stable transaction snapshot; concurrent update conflicts can abort transactions. Recheck whole transaction semantics.

CHAPTER 16 • ISOLATION AND MVCC

QUESTION Q078

How does MySQL InnoDB Repeatable Read differ from a generic slogan?

SHORT INTERVIEW ANSWER

Consistent reads typically use a transaction snapshot, while locking reads and writes use additional current-read/locking behavior.

TRACE THE EXAMPLE

```
Ordinary SELECT snapshot != SELECT ... FOR UPDATE semantics
```



WHY THIS WORKS

InnoDB's default Repeatable Read and PostgreSQL's default Read Committed are distinct baselines. Gap/next-key locks and access paths affect locking operations. Explain the exact statements and indexes instead of claiming every read sees an identical universe.

WATCH OUT

Mixing consistent reads and writes needs engine-specific reasoning.

FOLLOW-UP QUESTION + ANSWER

Which read establishes the InnoDB snapshot?

Under InnoDB Repeatable Read, the first consistent (nonlocking) read typically establishes the transaction read view, subject to transaction setup.

CHAPTER 16 • ISOLATION AND MVCC

QUESTION Q079

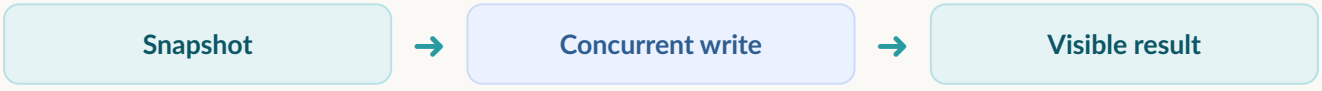
What is MVCC?

SHORT INTERVIEW ANSWER

Multi-version concurrency control lets readers use suitable row versions while writers manage changes and visibility under engine rules.

TRACE THE EXAMPLE

```
Reader snapshot -> visible older version
Writer commits newer version for later applicable snapshots
```



WHY THIS WORKS

MVCC can reduce reader/writer blocking but does not eliminate every lock or conflict. Old versions need cleanup and long snapshots can retain them. Foreign keys, DDL and explicit locking add other interactions.

WATCH OUT

MVCC is not lock-free execution for all SQL.

FOLLOW-UP QUESTION + ANSWER

What does a long report do to version cleanup?

Long snapshots retain obsolete versions or delay cleanup; monitor long transactions, undo/history in InnoDB, and vacuum dead tuples in PostgreSQL.

CHAPTER 16 • ISOLATION AND MVCC

QUESTION Q080

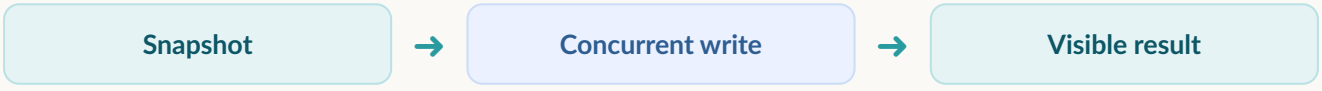
What does Serializable require from the application?

SHORT INTERVIEW ANSWER

It targets results equivalent to some serial execution, but the application must handle engine-specific blocking/conflict failures and retries.

TRACE THE EXAMPLE

```
Two concurrent decisions conflict -> one may need retry
Retry the complete transaction with a bounded policy
```



WHY THIS WORKS

PostgreSQL uses serialization-conflict detection; MySQL InnoDB has its own stronger locking behavior for this level. Preserve idempotency for external effects around retries. Serializable is a correctness choice, not a promise of no exceptions.

WATCH OUT

Retrying just the final statement may reuse a stale decision.

FOLLOW-UP QUESTION + ANSWER

Which work must be repeated together?

On serialization failure, retry the entire transaction with all dependent reads and writes, bounded attempts and idempotent external effects.

CHAPTER 17 • LOCKS, DEADLOCKS AND CONCURRENCY CONTROL

QUESTION Q081

What does SELECT FOR UPDATE do?

SHORT INTERVIEW ANSWER

It acquires locking-read protection on qualifying rows under engine semantics until the transaction releases it.

TRACE THE EXAMPLE

```
BEGIN;
SELECT available FROM inventory WHERE item_id=:id FOR UPDATE;
-- validate and update within this transaction
COMMIT;
```

Read / lock → Conflict handling → Commit

WHY THIS WORKS

Exact lock scope depends on engine, access path and isolation. It does not universally lock all future rows matching any predicate. Use constraints and guarded writes where simpler. Keep lock duration bounded.

WATCH OUT

Locking an existing row is not a universal solution for absence checks.

FOLLOW-UP QUESTION + ANSWER

What if the row does not exist?

Locking a missing row may not protect absence under every engine/isolation. Use a unique constraint, insert claim, advisory lock, or proper range-lock protocol.

CHAPTER 17 • LOCKS, DEADLOCKS AND CONCURRENCY CONTROL

QUESTION Q082

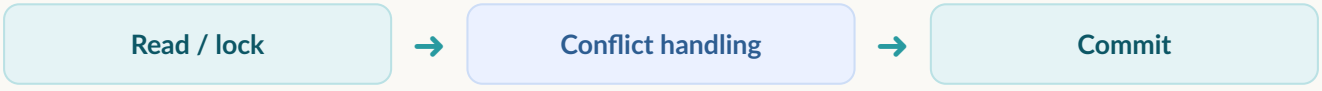
How do optimistic and pessimistic concurrency differ?

SHORT INTERVIEW ANSWER

Optimistic control detects conflicting state/version at write time; pessimistic control coordinates through locks before completing the operation.

TRACE THE EXAMPLE

```
UPDATE orders SET status=:new,version=version+1
WHERE order_id=:id AND version=:expected;
```



WHY THIS WORKS

Zero affected rows means conflict/missing under the contract. Optimistic approaches suit certain lower-contention workflows; repeated conflicts can make them expensive. Pessimistic locks can avoid wasted work but increase waits/deadlock risk.

WATCH OUT

Versioning is ineffective if other writers bypass the version rule.

FOLLOW-UP QUESTION + ANSWER

Which option fits a highly contended stock item?

A highly contended stock item usually needs a short guarded atomic UPDATE (or controlled pessimistic lock); optimistic retries may thrash under heavy contention.

CHAPTER 17 • LOCKS, DEADLOCKS AND CONCURRENCY CONTROL

QUESTION Q083

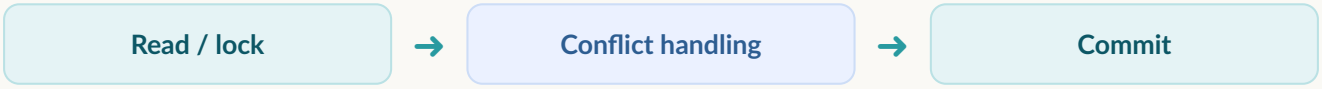
What causes a deadlock and how do you reduce it?

SHORT INTERVIEW ANSWER

Transactions hold resources while waiting cyclically for each other; consistent lock order and short transactions reduce opportunities.

TRACE THE EXAMPLE

```
T1 locks account 1 -> waits for 2
T2 locks account 2 -> waits for 1
```



WHY THIS WORKS

The database detects/handles victims under engine rules. Retry a failed transaction with bounds/backoff and safe effect identity. Indexes and statement ordering can influence lock footprints. Inspect deadlock evidence rather than only raising timeouts.

WATCH OUT

Deadlock and ordinary long lock wait are different.

FOLLOW-UP QUESTION + ANSWER

How would you impose a transfer lock order?

Always acquire account locks in deterministic ascending account_id order; keep transactions short and retry the entire failed transfer on deadlock.

CHAPTER 17 • LOCKS, DEADLOCKS AND CONCURRENCY CONTROL

QUESTION Q084

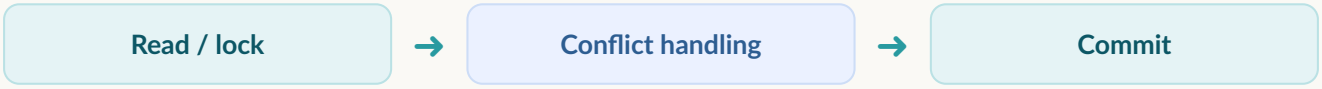
What is SKIP LOCKED useful for?

SHORT INTERVIEW ANSWER

It lets workers skip currently locked qualifying rows for queue-like claiming, with engine/version-specific syntax.

TRACE THE EXAMPLE

```
SELECT job_id FROM jobs WHERE state='READY'
ORDER BY job_id LIMIT 10 FOR UPDATE SKIP LOCKED;
```



WHY THIS WORKS

This is not a consistent general-purpose report: locked rows are deliberately omitted. Claim/update in the same transaction, then define lease/recovery for workers that die after claim. Avoid starvation and keep batches bounded.

WATCH OUT

Skipping a row does not mean the work was completed.

FOLLOW-UP QUESTION + ANSWER

How do you recover abandoned jobs?

Persist a claimed/lease state with deadline and unique job identity; a worker can requeue timed-out claims and processing remains idempotent.

CHAPTER 17 • LOCKS, DEADLOCKS AND CONCURRENCY CONTROL

QUESTION Q085

How do unique constraints support idempotent processing?

SHORT INTERVIEW ANSWER

They atomically reject duplicate identities while business effects commit in the same durable transaction.

TRACE THE EXAMPLE

```
processed_event(consumer_name,event_id) PRIMARY KEY
Insert identity + apply effect + commit together
```

Read / lock → Conflict handling → Commit

WHY THIS WORKS

Handle engine-specific conflict/transaction behavior correctly. A precheck or Redis flag stored separately leaves a failure gap. The key must remain stable across retries, and record cleanup must respect replay policy.

WATCH OUT

Unique event identity alone does not atomically protect a remote provider call.

FOLLOW-UP QUESTION + ANSWER

How do you return the prior result for a repeated API request?

Persist the request ID and immutable request fingerprint with status/result. On duplicate, return the stored result, or explicit in-progress/unknown status.

CHAPTER 18 • VIEWS, ROUTINES, PARTITIONING AND MAINTENANCE

QUESTION Q086

How do views and materialized views differ?

SHORT INTERVIEW ANSWER

An ordinary view is a stored query definition; a materialized view stores results that require a refresh/maintenance strategy.

TRACE THE EXAMPLE

```
CREATE VIEW paid_orders AS SELECT * FROM orders WHERE status='PAID';
```

Storage policy → Maintenance → Read model

WHY THIS WORKS

PostgreSQL offers native materialized views; MySQL does not offer the same native materialized-view feature. Views do not automatically index or cache their results. Updatability and security behavior depend on the definition/engine.

WATCH OUT

A view can still execute expensive joins for every request.

FOLLOW-UP QUESTION + ANSWER

When would a maintained summary table help?

Maintain a summary table when live aggregation is too costly and updates can be controlled; define refresh lag, atomic replacement and reconciliation.

CHAPTER 18 • VIEWS, ROUTINES, PARTITIONING AND MAINTENANCE

QUESTION Q087

How do stored procedures and functions differ?

SHORT INTERVIEW ANSWER

They are database-side routines with engine-specific invocation, return, side-effect and transaction rules.

TRACE THE EXAMPLE

```
Function used as an expression where supported
Procedure invoked with CALL under engine rules
```

Storage policy → Maintenance → Read model

WHY THIS WORKS

Do not assert functions can never mutate or procedures always commit: semantics differ across PostgreSQL/MySQL and languages. Centralized routines can reduce round trips but introduce deployment/test ownership. Document boundaries.

WATCH OUT

A routine does not automatically protect invariants better than ordinary SQL.

FOLLOW-UP QUESTION + ANSWER

Who versions and tests the routine?

Database routines should be versioned in migrations, reviewed with application code, and tested on the target engine with rollback practices.

CHAPTER 18 • VIEWS, ROUTINES, PARTITIONING AND MAINTENANCE

QUESTION Q088

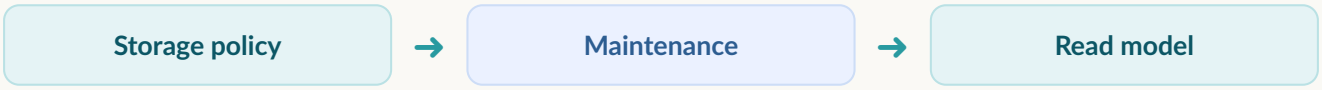
What are triggers useful for and what are their costs?

SHORT INTERVIEW ANSWER

Triggers react to supported data events and can enforce/maintain selected behavior, but introduce less visible work and ordering/error complexity.

TRACE THE EXAMPLE

```
INSERT order -> configured trigger maintains an audit row
```



WHY THIS WORKS

Review per-row/per-statement support, transaction behavior and recursion. A durable audit needs clear identity and access policy. Avoid making remote calls or hidden cross-table workflows without controlled failure semantics.

WATCH OUT

The application's one INSERT can cause more work than the visible statement suggests.

FOLLOW-UP QUESTION + ANSWER

How would you trace a trigger-induced failure?

Log the database exception and statement/trigger context without secrets, inspect trigger definitions and reproduce with a small transaction.

CHAPTER 18 • VIEWS, ROUTINES, PARTITIONING AND MAINTENANCE

QUESTION Q089

How does table partitioning differ from indexing and sharding?

SHORT INTERVIEW ANSWER

Partitioning divides a logical table into managed pieces; indexing creates access paths; sharding distributes data across independent database boundaries.

TRACE THE EXAMPLE

```
Partition orders by ordered_at range
Index each relevant access path
Shard tenants across databases only with an explicit routing design
```

Storage policy → Maintenance → Read model

WHY THIS WORKS

Partition pruning helps only when predicates and bounds permit it. Constraints, uniqueness, joins and maintenance differ by engine. More partitions are not automatically faster. Sharding adds cross-boundary consistency and operations costs.

WATCH OUT

Partitioning does not replace all indexes.

FOLLOW-UP QUESTION + ANSWER

Which query can prune to one month?

A half-open time range on the partition key, e.g. ordered_at >= month_start AND ordered_at < next_month, allows pruning when partitions align.

CHAPTER 18 • VIEWS, ROUTINES, PARTITIONING AND MAINTENANCE

QUESTION Q090

Why do statistics and version cleanup matter?

SHORT INTERVIEW ANSWER

The optimizer needs distribution evidence, and MVCC storage needs cleanup to avoid uncontrolled old-version accumulation.

TRACE THE EXAMPLE

```
PostgreSQL: ANALYZE / VACUUM / autovacuum
MySQL InnoDB: statistics and purge/history mechanisms
```

Storage policy → Maintenance → Read model

WHY THIS WORKS

Maintenance tasks have different roles; ordinary PostgreSQL VACUUM does not generally shrink every table file to the minimum size. Diagnose long transactions, growth and stale estimates. Avoid treating every engine's maintenance commands as interchangeable.

WATCH OUT

A single maintenance command is not a universal slow-query fix.

FOLLOW-UP QUESTION + ANSWER

What evidence indicates old snapshots are delaying cleanup?

PostgreSQL: long-running transactions/replication slots and retained dead tuples; MySQL: long-running read views and growing InnoDB history list length.

CHAPTER 19 • APPLICATION SQL, SECURITY AND PAGINATION

QUESTION Q091

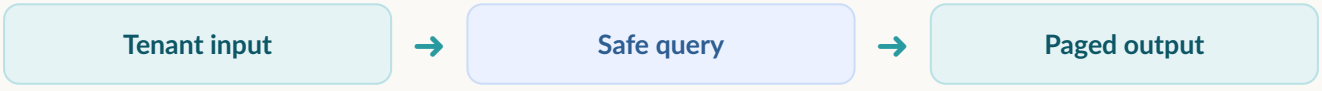
How do parameterized queries prevent SQL injection?

SHORT INTERVIEW ANSWER

They bind values separately from SQL structure rather than concatenating untrusted text into executable syntax.

TRACE THE EXAMPLE

```
SELECT * FROM customers WHERE customer_id = ?
-- Bind a value through JDBC; do not concatenate request text.
```



WHY THIS WORKS

Placeholders generally represent values, not arbitrary table names or ORDER BY identifiers. Choose dynamic identifiers from an allow-list and keep privilege minimal. ORM usage is not proof every query path is parameterized.

WATCH OUT

Concatenating a sort expression remains unsafe.

FOLLOW-UP QUESTION + ANSWER

How do you allow multiple safe sort columns?

Use an allowlist mapping user sort choices to fixed column names and ASC/DESC tokens; bind values, never raw SQL identifiers.

CHAPTER 19 • APPLICATION SQL, SECURITY AND PAGINATION

QUESTION Q092

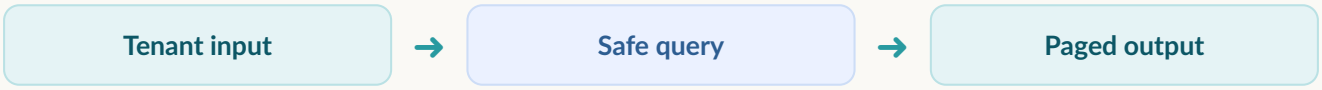
How do you prevent cross-tenant SQL leakage?

SHORT INTERVIEW ANSWER

Constrain access through authenticated tenant identity and enforce it consistently across reads, writes and relationship checks.

TRACE THE EXAMPLE

```
SELECT * FROM orders WHERE tenant_id=:trustedTenant AND order_id=:id;
```



WHY THIS WORKS

A globally unique order ID does not remove authorization requirements. Composite constraints and database row-level policies can help where supported. Dynamic/report/admin paths need deliberate ownership rules too.

WATCH OUT

A tenantId from the request is not automatically trusted.

FOLLOW-UP QUESTION + ANSWER

What happens when joining tenant-scoped tables?

Join using both tenant_id and entity ID, keep tenant predicates/row-level security enforced, and test crossed-ID attacks.

CHAPTER 19 • APPLICATION SQL, SECURITY AND PAGINATION

QUESTION Q093

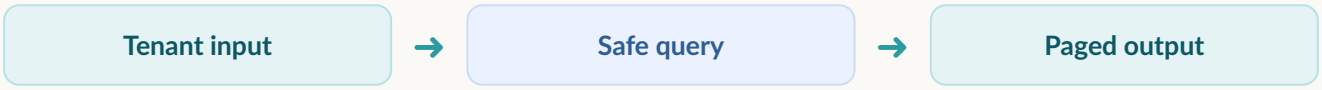
What is the N+1 query problem?

SHORT INTERVIEW ANSWER

One initial query causes many additional per-row queries, increasing round trips and total work.

TRACE THE EXAMPLE

```
Load 100 orders -> 100 separate customer/item lookups
```



WHY THIS WORKS

Use appropriate joins, batching or staged fetching, while respecting result cardinality and pagination. Joining several collections can create a different row-explosion problem. Count queries and inspect actual rows, not only ORM method names.

WATCH OUT

One giant join is not always the correct N+1 fix.

FOLLOW-UP QUESTION + ANSWER

How do you page orders while fetching their items?

First page a stable list of order IDs, then fetch items for those IDs in one query or batch; assemble results without multiplying page rows.

CHAPTER 19 • APPLICATION SQL, SECURITY AND PAGINATION

QUESTION Q094

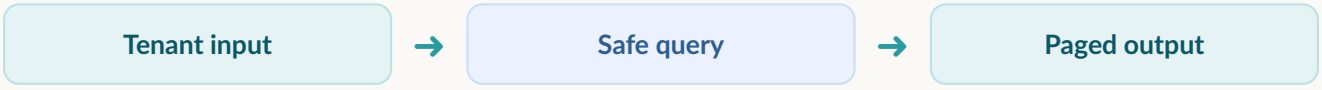
How does keyset pagination work?

SHORT INTERVIEW ANSWER

It requests rows after the last ordered key rather than skipping a growing offset.

TRACE THE EXAMPLE

```
SELECT order_id,ordered_at FROM orders
WHERE ordered_at<:lastTime OR
 (ordered_at=:lastTime AND order_id<:lastId)
ORDER BY ordered_at DESC,order_id DESC LIMIT 20;
```



WHY THIS WORKS

Use a stable unique tie-breaker, matching sort directions and useful index. Null ordering and mutable sort values require explicit handling. It improves deep navigation but does not promise one immutable snapshot across requests.

WATCH OUT

Changing an ordering value can move a row across the cursor.

FOLLOW-UP QUESTION + ANSWER

What if the user needs random page numbers?

Use offset pagination for bounded random access, or page-number metadata plus a stable snapshot; seek pagination is cursor-based by nature.

CHAPTER 19 • APPLICATION SQL, SECURITY AND PAGINATION

QUESTION Q095

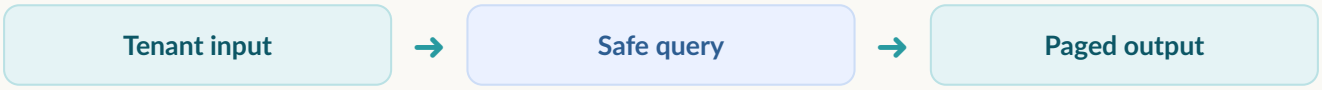
How should connection pools and query deadlines be designed?

SHORT INTERVIEW ANSWER

Bound concurrency, wait time and transaction/query duration according to database capacity and application SLOs.

TRACE THE EXAMPLE

```
Pool wait budget + query budget + response work <= request budget
```



WHY THIS WORKS

More connections can increase contention rather than throughput. A connection must return in clean transaction/session state. Cancellation/timeout can leave an uncertain write outcome, so correct identity and reconciliation matter.

WATCH OUT

Client timeout does not prove a database write rolled back.

FOLLOW-UP QUESTION + ANSWER

What should the API do after an unknown commit outcome?

Reconcile commit outcome via stable request identity before repeating work; return a pending/unknown status rather than falsely claiming failure.

CHAPTER 20 • SENIOR SCENARIOS AND QUERY-REVIEW DISCIPLINE

QUESTION Q096

A revenue query doubled totals. Where do you investigate?

SHORT INTERVIEW ANSWER

Check join grain and detail multiplicity before blaming SUM or the database.

TRACE THE EXAMPLE

```
orders -> order_items multiplies order amount per item
Aggregate item totals or avoid unnecessary detail join
```

Correctness → Plan evidence → Failure recovery

WHY THIS WORKS

Compare counts per order and trace a tiny counterexample. Aggregating each many-side before joining can restore the desired grain. Add constraints where one-to-one assumptions are real. Preserve equal-valued legitimate orders.

WATCH OUT

SUM(DISTINCT order.amount) is not a general repair.

FOLLOW-UP QUESTION + ANSWER

Which table owns the revenue fact?

The orders header owns order-level booked revenue; order_items owns line-level revenue if that is the metric. Define grain before summing.

CHAPTER 20 • SENIOR SCENARIOS AND QUERY-REVIEW DISCIPLINE

QUESTION Q097

A second-highest salary answer is disputed. What do you clarify?

SHORT INTERVIEW ANSWER

Ask whether the requirement is the second row, second distinct value, rank position or all employees at that value.

TRACE THE EXAMPLE

```
100,100,90 -> second row 100; second distinct salary 90
```

Correctness → Plan evidence → Failure recovery

WHY THIS WORKS

Specify null treatment and no-second-value behavior. Choose ROW_NUMBER, RANK, DENSE_RANK or scalar MAX accordingly. Tests with ties are essential because simple all-distinct data hides mistakes.

WATCH OUT

A syntactically valid query can solve the wrong ranking question.

FOLLOW-UP QUESTION + ANSWER

What should an empty result mean?

No second level should return NULL for a scalar aggregate, or zero rows for a filtered employee list; specify the API requirement.

CHAPTER 20 • SENIOR SCENARIOS AND QUERY-REVIEW DISCIPLINE

QUESTION Q098

The same query is slow for one tenant. What is likely?

SHORT INTERVIEW ANSWER

Data skew, cardinality estimates, access paths, plan reuse or contention may differ for that parameter.

TRACE THE EXAMPLE

```
Small tenant=100 rows; large tenant=10 million rows
Same text does not imply same optimal plan
```

Correctness → Plan evidence → Failure recovery

WHY THIS WORKS

Compare representative plans and waits with real parameters, statistics and index distribution. Do not force an index globally without evaluating smaller and larger workloads. Use tenancy/partitioning only if requirements justify it.

WATCH OUT

One fast benchmark does not prove all parameter cases.

FOLLOW-UP QUESTION + ANSWER

How would you measure a plan regression?

Capture representative actual plans, buffers, row estimates, latency by tenant and parameter, before/after deploy. Compare statistics and selectivity.

CHAPTER 20 • SENIOR SCENARIOS AND QUERY-REVIEW DISCIPLINE

QUESTION Q099

How would you design a safe bank transfer?

SHORT INTERVIEW ANSWER

Enforce account validity/funds invariants in one reviewed transaction, lock or guard writes coherently, and use stable transfer identity for retries.

TRACE THE EXAMPLE

```
Transaction: validate both accounts -> debit -> credit -> ledger -> commit
Consistent account lock order; same currency/amount contract
```

Correctness → Plan evidence → Failure recovery

WHY THIS WORKS

Check affected rows and constraints. External notifications publish from durable intent after the database outcome. A timeout after commit is an uncertain result, so reusing the transfer identity should return/reconcile the existing outcome.

WATCH OUT

Two balance updates alone omit important validation and idempotency.

FOLLOW-UP QUESTION + ANSWER

What if the destination does not exist?

Reject/rollback the full transaction if destination row is missing; constrain accounts and verify affected row counts before committing.

CHAPTER 20 • SENIOR SCENARIOS AND QUERY-REVIEW DISCIPLINE

QUESTION Q100

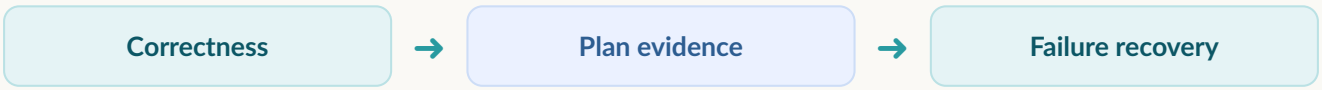
What makes a strong senior SQL interview answer?

SHORT INTERVIEW ANSWER

State the result grain, NULL/tie policy, invariants, dialect, query, expected output and performance/concurrency evidence.

TRACE THE EXAMPLE

```
Correctness: edge-case fixture and expected rows
Performance: representative plan and measurements
Concurrency: two-session schedule and recovery policy
```



WHY THIS WORKS

A small query with correct semantics is better than clever syntax solving the wrong problem. Explain assumptions and compare alternatives from evidence. Distinguish portable query checks from engine-specific transaction/optimizer verification.

WATCH OUT

Passing one happy-path dataset is not sufficient proof.

FOLLOW-UP QUESTION + ANSWER

Which counterexample would you add first?

Try NULLs and duplicates in a one-to-many JOIN first; they expose many incorrect SQL answers very quickly.

PRACTICE LAB

15 worked SQL challenges

Each exercise includes the original SQL, expected results from the shared fixture, an explanation and one extension challenge.

RUN THE LABS

Install Python 3. Run **python3 verify_all.py** in the coding ZIP. All 15 SELECT queries use the same disposable SQLite fixture.

C01	Second-highest distinct salary
C02	Top two salary levels per department
C03	Customers with no orders
C04	Employee count for every department
C05	Paid revenue including customers with zero
C06	Running paid total per customer
C07	Previous paid amount and difference
C08	Employees above their department average
C09	Latest order per customer
C10	Revenue without detail-join inflation

THE LAST FOUR

Complete the practical track

C12	Consecutive integer islands
C13	Bounded organization traversal
C14	Seek pagination with tied dates
C15	Conditional order-status report

REPRODUCIBILITY

The source ZIP includes 15 SQL queries, a populated SQL fixture, 15 result CSVs, instructions and a Python verification runner. Results on other databases may display types/dates differently.

WHAT IS NOT VERIFIED BY SQLITE

PostgreSQL/MySQL locking, DDL, isolation, optimizer plans and exact decimal behavior are outside this local SELECT test.

SQL WORKED LAB C01

Second-highest distinct salary

YOUR TASK

Return one row containing the second-highest known salary value. Tied employees count as one salary level.

WORKED SQL SOLUTION

```
SELECT MAX(salary) AS second_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

EXPECTED RESULTS • 1 ROWS

second_salary
90000

WHY IT WORKS

The inner MAX is 100000. The outer MAX of lower values is 90000. Unlike selecting the second row, duplicates cannot change the distinct-level meaning. An aggregate over no qualifying rows still returns one row with NULL.

TRY NEXT

Try no employees, all NULL salaries, and only one distinct salary: each should return one NULL. If asked to return people at that level, join/filter by the resulting salary instead.

SQL WORKED LAB C02

Top two salary levels per department

YOUR TASK

Return all employees in the two highest distinct salary levels of each assigned department. Include ties and exclude unknown salaries.

WORKED SQL SOLUTION

```
WITH ranked AS (
  SELECT department_id, employee_id, name, salary,
    DENSE_RANK() OVER (
      PARTITION BY department_id ORDER BY salary DESC
    ) AS salary_rank
  FROM employees
  WHERE department_id IS NOT NULL AND salary IS NOT NULL
)
SELECT department_id, employee_id, name, salary, salary_rank
FROM ranked WHERE salary_rank <= 2
ORDER BY department_id, salary_rank, employee_id;
```

EXPECTED RESULTS • 5 ROWS

department_id	employee_id	name	salary	salary_rank
10	1	Sai	100000	1
10	2	Meena	80000	2
10	3	Arun	80000	2
20	4	Priya	90000	1
20	5	Ravi	60000	2

WHY IT WORKS

Engineering returns three people because Meena and Arun share the second salary level. The outer employee_id ordering makes presentation deterministic without destroying the salary ties in DENSE_RANK.

TRY NEXT

For exactly two people, use ROW_NUMBER ordered by salary DESC, employee_id. Test a department with one salary level and three employees at the highest salary.

SQL WORKED LAB C03

Customers with no orders

YOUR TASK

Return customers with no order of any status. Preserve the meaning if the orders table is empty.

WORKED SQL SOLUTION

```
SELECT c.customer_id, c.name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

EXPECTED RESULTS • 1 ROWS

customer_id	name
4	Priya

WHY IT WORKS

Priya has no matching order. EXISTS asks about matching rows rather than the value selected inside the subquery. To ask about no paid orders, add the paid predicate inside it.

TRY NEXT

An empty orders table should return every customer. A duplicate qualifying child does not duplicate a customer in this result.

SQL WORKED LAB C04

Employee count for every department

YOUR TASK

Include empty departments and count matched employees, not null-extended rows.

WORKED SQL SOLUTION

```
SELECT d.department_id, d.name,
       COUNT(e.employee_id) AS employees_count
FROM departments d
LEFT JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.name
ORDER BY d.department_id;
```

EXPECTED RESULTS • 3 ROWS

department_id	name	employees_count
10	Engineering	3
20	Sales	2
30	HR	0

WHY IT WORKS

HR creates one null-extended join row; COUNT(e.employee_id) ignores its NULL. COUNT(*) would give HR a misleading count of one. Nila has no assigned department and does not belong to these department groups.

TRY NEXT

Add a salary requirement to ON when you must keep empty departments. Check both COUNT(*) and COUNT(e.employee_id) before and after the change.

SQL WORKED LAB C05

Paid revenue including customers with zero

YOUR TASK

Return one paid order-header total per customer; include customers with no qualifying order.

WORKED SQL SOLUTION

```
WITH paid AS (
  SELECT customer_id, SUM(amount) AS revenue
  FROM orders WHERE status = 'PAID'
  GROUP BY customer_id
)
SELECT c.customer_id, c.name,
       COALESCE(p.revenue, 0) AS revenue
FROM customers c
LEFT JOIN paid p ON p.customer_id = c.customer_id
ORDER BY c.customer_id;
```

EXPECTED RESULTS • 4 ROWS

customer_id	name	revenue
1	Sai	350
2	Meena	270
3	Arun	80
4	Priya	0

WHY IT WORKS

The CTE establishes one row per customer before joining. Its total is 700 across all paid orders, including the February order. Replacing absent paid revenue with zero is explicit here.

TRY NEXT

Add a half-open date predicate inside the CTE for a reporting period. January paid revenue is 630; do not accidentally include order 108.

SQL WORKED LAB C06

Running paid total per customer

YOUR TASK

Return each paid order and the cumulative amount for that customer in date/ID order.

WORKED SQL SOLUTION

```
SELECT customer_id, order_id, amount,
  SUM(amount) OVER (
    PARTITION BY customer_id ORDER BY ordered_at, order_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total
FROM orders WHERE status = 'PAID'
ORDER BY customer_id, ordered_at, order_id;
```

EXPECTED RESULTS • 6 ROWS

customer_id	order_id	amount	running_total
1	101	100	100
1	102	150	250
1	106	100	350
2	103	200	200
2	108	70	270
3	105	80	80

WHY IT WORKS

PARTITION BY resets the total per customer. ROWS and a unique order define a row-by-row progression. Filtering before the window means pending/cancelled orders are absent from both rows and totals.

TRY NEXT

Insert two paid orders on the same date and verify both steps. Removing the ID tie-breaker or relying on a default peer-aware frame can change the intended result.

SQL WORKED LAB C07

Previous paid amount and difference

YOUR TASK

Compare each paid order amount with the previous paid order for the same customer.

WORKED SQL SOLUTION

```
WITH history AS (
  SELECT customer_id, order_id, ordered_at, amount,
    LAG(amount) OVER (
      PARTITION BY customer_id ORDER BY ordered_at, order_id
    ) AS previous_amount
  FROM orders WHERE status = 'PAID'
)
SELECT customer_id, order_id, previous_amount,
       amount - previous_amount AS difference
FROM history
ORDER BY customer_id, ordered_at, order_id;
```

EXPECTED RESULTS • 6 ROWS

customer_id	order_id	previous_amount	difference
1	101	NULL	NULL
1	102	100	50
1	106	150	-50
2	103	NULL	NULL
2	108	200	-130
3	105	NULL	NULL

WHY IT WORKS

The first paid order has no previous paid row, so its previous value and arithmetic difference are NULL. Using zero as the default would answer a different question.

TRY NEXT

Try one-row partitions and same-day ties. Clarify whether previous means previous paid order or previous order of any status.

SQL WORKED LAB C08

Employees above their department average

YOUR TASK

Return assigned employees whose known salary is strictly greater than their department average.

WORKED SQL SOLUTION

```
WITH averages AS (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM employees WHERE department_id IS NOT NULL
  GROUP BY department_id
)
SELECT e.employee_id, e.name, e.department_id, e.salary
FROM employees e
JOIN averages a ON a.department_id = e.department_id
WHERE e.salary > a.avg_salary
ORDER BY e.employee_id;
```

EXPECTED RESULTS • 2 ROWS

employee_id	name	department_id	salary
1	Sai	10	100000
4	Priya	20	90000

WHY IT WORKS

Engineering averages about 86666.67 and Sales averages 75000. AVG excludes NULL salaries. A department with only NULL salaries has a NULL average, so no employee qualifies via a normal comparison.

TRY NEXT

Test an employee exactly at the average, and a department containing one known salary. Neither should qualify under the strict greater-than rule.

SQL WORKED LAB C09

Latest order per customer

YOUR TASK

Return the latest order of any status for each customer who has orders. Higher order_id breaks equal-date ties.

WORKED SQL SOLUTION

```
WITH ranked AS (
  SELECT customer_id, order_id, ordered_at, status,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY ordered_at DESC, order_id DESC
    ) AS rn
  FROM orders
)
SELECT customer_id, order_id, ordered_at, status
FROM ranked WHERE rn = 1
ORDER BY customer_id;
```

EXPECTED RESULTS • 3 ROWS

customer_id	order_id	ordered_at	status
1	106	2026-01-04	PAID
2	108	2026-02-01	PAID
3	107	2026-01-05	PENDING

WHY IT WORKS

ROW_NUMBER picks one complete row. MAX(date) joined back without a tie rule can return multiple rows. Priya is absent because this query starts from orders; left join the rn=1 result to customers to preserve her.

TRY NEXT

Add equal-date orders for a customer. Confirm the chosen ID and decide whether the product actually requires all latest-date ties.

SQL WORKED LAB C10

Revenue without detail-join inflation

YOUR TASK

Compute paid header revenue and paid item value. Collapse item rows to one row per order before joining.

WORKED SQL SOLUTION

```
WITH item_totals AS (
  SELECT order_id, SUM(quantity * unit_price) AS item_value
  FROM order_items GROUP BY order_id
)
SELECT SUM(o.amount) AS header_revenue,
       SUM(COALESCE(i.item_value, 0)) AS item_value
FROM orders o
LEFT JOIN item_totals i ON i.order_id = o.order_id
WHERE o.status = 'PAID';
```

EXPECTED RESULTS • 1 ROWS

header_revenue	item_value
700	700

WHY IT WORKS

Order 101 has two item rows. Summing its header amount after a raw item join produces 800 instead of 700. SUM(DISTINCT amount) gives 600 here because different orders can both cost 100. Pre-aggregation restores the intended grain.

TRY NEXT

Add an order with no item rows and clarify whether that is valid. Item totals need not equal header totals in a real system with taxes, fees and discounts.

SQL WORKED LAB C11

Deterministic duplicate candidates

YOUR TASK

For each non-null exact email, keep the smallest contact_id and return the other IDs. Do not delete anything yet.

WORKED SQL SOLUTION

```
WITH ranked AS (
  SELECT contact_id, email,
    ROW_NUMBER() OVER (
      PARTITION BY email ORDER BY contact_id
    ) AS rn
  FROM contacts WHERE email IS NOT NULL
)
SELECT contact_id, email FROM ranked
WHERE rn > 1 ORDER BY contact_id;
```

EXPECTED RESULTS • 1 ROWS

contact_id	email
2	a@example.com

WHY IT WORKS

Only ID 2 is a duplicate candidate. This fixture uses exact lowercase values; production equality depends on collation and any normalization policy. Missing emails are excluded by the stated requirement.

TRY NEXT

Test three equal emails, case variants and multiple NULLs. Before deletion, review survivor rules, foreign-key references and concurrent inserts; then enforce the chosen uniqueness rule.

SQL WORKED LAB C12

Consecutive integer islands

YOUR TASK

Find consecutive groups of distinct integers and return their start, end and length.

WORKED SQL SOLUTION

```
WITH distinct_values AS (
  SELECT DISTINCT n FROM sequence_values
), numbered AS (
  SELECT n, n - ROW_NUMBER() OVER (ORDER BY n) AS island_key
  FROM distinct_values
)
SELECT MIN(n) AS start_n, MAX(n) AS end_n, COUNT(*) AS length
FROM numbered GROUP BY island_key
ORDER BY start_n;
```

EXPECTED RESULTS • 3 ROWS

start_n	end_n	length
1	3	3
5	6	2
9	9	1

WHY IT WORKS

For consecutive integers, both n and row number increase by one, so their difference stays constant. DISTINCT removes duplicate integers before numbering. Date-based islands require dialect-aware date arithmetic.

TRY NEXT

Insert a duplicate 2 and verify the same groups. Try an empty table, one integer and negative integers; review arithmetic range when real values approach type limits.

SQL WORKED LAB C13

Bounded organization traversal

YOUR TASK

Starting at Sai (employee 1), list the reporting tree with depth. The fixture is acyclic; limit depth to ten.

WORKED SQL SOLUTION

```
WITH RECURSIVE org AS (
  SELECT employee_id, name, manager_id, 0 AS depth
  FROM employees WHERE employee_id = 1
  UNION ALL
  SELECT e.employee_id, e.name, e.manager_id, o.depth + 1
  FROM employees e JOIN org o ON e.manager_id = o.employee_id
  WHERE o.depth < 10
)
SELECT employee_id, name, depth FROM org
ORDER BY depth, employee_id;
```

EXPECTED RESULTS • 3 ROWS

employee_id	name	depth
1	Sai	0
2	Meena	1
3	Arun	1

WHY IT WORKS

The anchor emits Sai, and the recursive term follows employees whose manager is a current node. Priya/Ravi belong to another root. The depth bound stops unbounded expansion but may truncate a legitimate deeper tree.

TRY NEXT

A production graph needs a deliberate cycle policy and path/visited detection, not only a depth cap. Test a missing root, a leaf root and a cycle in a disposable fixture.

SQL WORKED LAB C14

Seek pagination with tied dates

YOUR TASK

Return the next three orders after cursor (2026-01-03, 105), sorted by date descending then ID descending.

WORKED SQL SOLUTION

```
SELECT order_id, ordered_at
FROM orders
WHERE ordered_at < '2026-01-03'
   OR (ordered_at = '2026-01-03' AND order_id < 105)
ORDER BY ordered_at DESC, order_id DESC
LIMIT 3;
```

EXPECTED RESULTS • 3 ROWS

order_id	ordered_at
104	2026-01-03
103	2026-01-02
102	2026-01-02

WHY IT WORKS

The cursor comparison includes both sort fields, so order 104 is not skipped despite sharing a date with 105. This avoids reading a deep offset, but it does not create a frozen snapshot across requests.

TRY NEXT

Use bound parameters in application SQL, include tenant/filter context in the cursor contract, and consider an index matching equality filters then sort keys. Test changed sort values and a deleted cursor row.

SQL WORKED LAB C15

Conditional order-status report

YOUR TASK

Return paid, pending and cancelled counts for every customer, including customers with no orders.

WORKED SQL SOLUTION

```
SELECT c.customer_id,
  SUM(CASE WHEN o.status = 'PAID' THEN 1 ELSE 0 END) AS paid,
  SUM(CASE WHEN o.status = 'PENDING' THEN 1 ELSE 0 END) AS pending,
  SUM(CASE WHEN o.status = 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id
ORDER BY c.customer_id;
```

EXPECTED RESULTS • 4 ROWS

customer_id	paid	pending	cancelled
1	3	0	0
2	2	0	1
3	1	1	0
4	0	0	0

WHY IT WORKS

Each CASE turns one stated condition into a 1/0 value, so the preserved row for Priya contributes zero in every column. These three statuses cover the fixture; a new status needs an explicit reporting decision.

TRY NEXT

Add a new status and an additional pending order. If detail joins are introduced, restore one row per order before counting.

LAST-MINUTE REVISION

10 traps interviewers test

01	= NULL is UNKNOWN: use IS NULL.
02	COUNT(*) after LEFT JOIN can count preserved empty rows.
03	LEFT JOIN + WHERE on right side can drop unmatched rows.
04	Join detail tables at the right grain to avoid duplicated totals.
05	Define tie policy for top-N and salaries.
06	Window ORDER BY is different from final display ordering.
07	An index does not always beat a scan.
08	DDL rollback depends on the engine and command.
09	MVCC does not eliminate write conflicts or locks.
10	Timeout does not prove a write failed.

THE FIVE TEST INPUTS

Empty input, missing match, duplicate value, NULL and one-to-many child rows. These catch many SQL mistakes.

30-DAY ROADMAP

Days 01–15

DAY 01	SQL grain / Q001–Q005
DAY 02	NULL + types / Q006–Q010
DAY 03	Constraints / Q011–Q015
DAY 04	Half-open dates / Q016–Q020
DAY 05	Joins / Q021–Q025
DAY 06	Grouping / Q026–Q030
DAY 07	Review first six chapters
DAY 08	EXISTS + NOT EXISTS / C03
DAY 09	CTEs + recursion / C13
DAY 10	Windows / C06, C07
DAY 11	Ranking / C01, C02
DAY 12	Deduplication / C11
DAY 13	Latest rows + islands / C09, C12

30-DAY ROADMAP

Days 16–30

DAY 16	Normalization / Q056–Q060
DAY 17	Indexes / Q061–Q065
DAY 18	Explain plans / Q066–Q070
DAY 19	ACID / Q071–Q075
DAY 20	MVCC + snapshots / Q076–Q080
DAY 21	Compare PostgreSQL vs MySQL
DAY 22	Locking + deadlocks / Q081–Q085
DAY 23	Views + maintenance / Q086–Q090
DAY 24	App SQL + injection / Q091–Q095
DAY 25	Keyset pagination / C14
DAY 26	Aggregation + revenue / C05,C10
DAY 27	Senior SQL cases / Q096–Q100
DAY 28	Mock interview: query round

INTERVIEW EVIDENCE

Practice like a senior engineer

Do more than define SQL terms. Explain the boundary between a query result, a transaction and a remote side effect.

MOCK INTERVIEW A • 45 MIN

Explain NULL and COUNT. Write second distinct salary (C01), top-two salary levels (C02), customers without orders (C03), and revenue without header duplication (C10).

MOCK INTERVIEW B • 60 MIN

Investigate a slow multi-tenant report: validate row grain and tenant isolation, compare real plans, explain index selectivity, choose transaction isolation, plan deadlock retries and reconcile unknown payment outcomes.

BEFORE YOU CALL IT DONE

Show result correctness with tiny edge cases. Show performance with an actual plan. Show concurrency safety with overlapping transactions on the chosen engine.

SOURCE & VERIFICATION

What is verified?

EXECUTED

All 15 read-only lab queries were run against the provided training fixture in SQLite and compared with saved expected CSV outputs.

SOURCE ALIGNMENT

All 100 interview question wordings, 100 follow-up prompts and 15 lab SQL solutions were taken from the attached 82-page guide. Follow-up answers and page layouts are newly written.

NEEDS TARGET DATABASE TESTING

PostgreSQL 17 and MySQL 8.4 locking, optimizer plans, DDL rollback, isolation and precise decimal/type behavior were not reproduced by this SQLite runner.

FURTHER READING

PostgreSQL 17: postgresql.org/docs/17/ • MySQL 8.4: dev.mysql.com/doc/refman/8.4/ • Java 17 JDBC PreparedStatement API.